

NetCore-Tetra Kompletguide

Installation der Basisstation und aller 25 LXC-Dienste - Konfiguration, Bedienung, Offline-Fallback und Abnahme

NetCore-Tetra Projekt

Stand: 27. September 2026

OPEN LAB: Viele Backend-WebUIs arbeiten ohne Login, Management-Token oder TLS. Die Warnzentrale verlangt dagegen ein Token. Die direkten Listener gehören in ein geschütztes Managementnetz; die Dienste können dort im selben VLAN liegen. Security Core und KMF benötigen besonders restriktive Zugriffsregeln.

Funkrecht: Frequenzen, Sendeleistung, Rufzeichen/Identitäten und Betriebsart müssen zur jeweiligen Genehmigung passen. Die Anleitung ersetzt weder Frequenzuteilung noch EMV-/HF-Abnahme.

Aktueller Quellenstand

Diese Ausgabe dokumentiert Release **v1.9.0** auf main bei 086a81fa8820ef579c475a65a38e3d23644c52f0. Stichtag ist der **27. September 2026 in Europe/Berlin**; die Veröffentlichung erfolgte am 26. September um 23:02 UTC. Die neue Warnzentrale erweitert das Open-Lab-Inventory auf **25 deploybare Dienste**. system-backend/services.toml zählt **26 Einträge einschließlich der Bibliothek shared**. Provisioning Core ist ein zusätzlicher Workspace-Dienst und steht in keiner dieser beiden Listen.

Gegenüber den bisherigen Vorlagen sind NINA/KATWARN, eigene Warnungen, GPS-gestützte Einzel-SDS, die Control-Room-Telemetrie vom Node Gateway sowie die SDS-, Call-Control-, Frame-18- und SIP-Reparaturen berücksichtigt. Bestehende Konfigurationen und Persistenzdaten bleiben beim Update erhalten. Ein erfolgreicher Build ersetzt keine Ende-zu-Ende- oder Funkabnahme.

Die Referenzprüfung ergab OK: 25 services, contract=netcore.v1, mode=open_lab. Der Full-System-Audit ergab **FAIL mit drei Befunden**: Die Prüfliste erwartet noch 24 Dienste; alert-service bietet die erwarteten /metrics- und /openapi.json-Marker nicht; die Root-TBS-Konfiguration verwendet für das Gateway 10.0.1.179, das Inventory 10.0.20.10. Der separat bestätigte GitHub-Workflow **Warning service** für diesen Commit ist erfolgreich. Diese Aussagen betreffen Quellstand, lokale statische Prüfung und CI; laufende TBS, LXC und Funkgeräte wurden nicht geprüft.

Inhalt

- - 1. Schnellstart und Zielbild
- - 2. Architektur, Dienste und Ports
- - 3. Planung von Netzwerk, LXC und Storage
- - 4. Installation der TETRA-Basisstation
- - 5. Konfiguration der Basisstation
- - 6. Proxmox-LXC vorbereiten
- - 7. Automatisches Deployment aller LXC
- - 8. Manuelle Installation und Bedienung je Dienst
- - 9. Konfigurationsreferenz der LXC-Dienste
- - 10. Betriebs- und Bedienabläufe
- - 11. Offline-Fallback der Basisstation
- - 12. Backup, Update und Wiederherstellung
- - 13. Systemtest und Abnahme
- - 14. Fehlersuche
- - 15. Inbetriebnahme-Checkliste
- Anhang A: Port-, Pfad- und Befehlsreferenz
- Anhang B: Warnzentrale und v1.9.0 Betrieb
- Anhang C: Roadmap und Releases

1. Schnellstart und Zielbild

Dieser Guide führt vom leeren Debian-/Raspberry-Pi-System und frisch angelegten Proxmox-LXC bis zur ersten vollständigen Laborabnahme. Die empfohlene Topologie besteht aus einer funkseitigen Basisstation und 25 voneinander getrennten Backend-LXC. Die Basisstation bleibt bei Ausfall von Internet, VPN oder Core-Diensten lokal betriebsfähig.

1.1 Empfohlene Reihenfolge

1. Management-VLAN, IP-Adressen, DNS/NTP und Storage planen.
2. Basisstation bauen und zunächst **standalone** mit lokaler Konfiguration und Fallback-Datei testen.
3. 25 Debian-LXC anlegen; IP Gateway erhält zusätzlich /dev/net/tun.
4. Korrigiertes Inventory aus diesem Guide anpassen und `validate`, `plan`, `render`, `apply --dry-run` ausführen.
5. LXC in Abhängigkeitsreihenfolge deployen.
6. Basisstation an den Node Gateway anbinden; zentrale SDS-Routen zunächst noch deaktiviert lassen.
7. Smoke-Test, dann funktionalen Full-Test, zuletzt Fault-/Fallback-Test durchführen.
8. Shadow-Dienste einzeln und kontrolliert auf authoritative schalten.

1.2 Minimale Startbefehle

Auf dem Deployment-Host

```
cp deploy/open-lab/inventory.example.toml deploy/open-lab/inventory.toml
${EDITOR:-nano} deploy/open-lab/inventory.toml
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml plan
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml render
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply --dry-run
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml status
```

Danach auf der Basisstation [control_room] auf den Node Gateway zeigen lassen und den Dienst neu starten. Der erste Systemtest ist:

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --profile smoke
```

2. Architektur, Dienste und Ports

2.1 Dienstmatrix

Reihenfolge	Dienst	Beispiel-IP	Port	Aufgabe	Abhängigkeiten
1	node-gateway	10.0.20.10	8080	Zentraler Einstiegspunkt für TBS-WebSockets und normalisierter Transport zu den Backend-Diensten.	-
2	mobility-core	10.0.20.11	8090	Teilnehmerlage, Serving-Node-Zuordnung und MM-Context-Transfers zwischen TBS.	node-gateway
3	subscriber-core	10.0.20.12	8100	Zentrale Teilnehmerprofile und Admission-Policy.	node-gateway
4	group-core	10.0.20.13	8110	GSSI-Stammdaten, Mitgliedschaften, Affiliationen und DGNA.	node-gateway, subscriber-core
5	call-control	10.0.20.14	8120	Netzweite logische Gruppen-/Individualrufe, Call Legs, Floor und Restore.	node-gateway, subscriber-core, group-core, mobility-core
6	media-switch	10.0.20.15	8130	Routing bereits codierter 35-Byte-TETRA-Sprachframes zwischen TBS-Call-Legs.	node-gateway, call-control
7	recorder	10.0.20.16	8140	Passive, verlustfreie Aufzeichnung von TACELP-Frames außerhalb des Rufpfads.	media-switch
8	sds-router	10.0.20.17	8150	Zentrale SDS-/Statusvermittlung, Store-and-forward und Anwendungsrouten.	node-gateway, subscriber-core, group-core, mobility-core
9	packet-core	10.0.20.18	8160	PDP-/NSAPI-State-Machine, IPv4-Leases, Mobility	node-gateway, subscriber-core, mobility-core

Reihenfolge	Dienst	Beispiel-IP	Port	Aufgabe	Abhängigkeiten
				Anchoring, Fragmente und Flow Control.	
10	ip-gateway	10.0.20.19	8170	Layer-3-Übergang zwischen Packet Core und normalen IPv4-Netzen.	packet-core
11	security-core	10.0.20.20	8180	Security-Class-Policy, Lab-Authentisierung, DCK-Kontexte, Sperren und Audit.	node-gateway, subscriber-core
12	kmf	10.0.20.21	8190	Lifecycle für CCK/GCK/SCK, Rotation, Crypto Periods und OTAR-Orchestrierung.	security-core
13	transit	10.0.20.22	8200	NetCore-native Regionalvermittlung mit Peers, Routen, Sessions und Failover.	mobility-core, call-control, media-switch, sds-router
14	application-gateway	10.0.20.23	8220	Adapter- und Workflow-Gateway für externe Anwendungen, Webhooks, Vorlagen und TTS.	sds-router
15	media-library	10.0.20.24	8230	Zentrale Medienablage, Vorschau, Freigabe, TACELP-Cache und Playout.	media-switch, recorder, application-gateway
16	control-room	10.0.20.25	9010	Zentrale Lage-, Operator-, Incident- und Schichtbuchebenen.	node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library
17	observability	10.0.20.26	8210	Zentrale Metriken, Logs, Traces, Alerts,	node-gateway, subscriber-core, group-core,

Reihenfolge	Dienst	Beispiel-IP	Port	Aufgabe	Abhängigkeiten
				Silences und Diagnosepakete.	mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library, control-room
18	iot-gateway	10.0.20.27	8240	MQTT, Home Assistant, Homematic und Command/Ack-Ledger	node-gateway, mobility-core, call-control, sds-router
19	hardware-gateway	10.0.20.28	8250	Hardware-I/O, Sensoren und begrenzte Aktorbefehle	iot-gateway
20	rf-monitor	10.0.20.29	8260	RF-Messwerte, Qualität und Diagnose	iot-gateway
21	alarm-workflow	10.0.20.30	8270	Alarmregeln, Ereignisse und Eskalation	iot-gateway, sds-router, hardware-gateway, rf-monitor
22	task-workflow	10.0.20.31	8280	Strukturierte Aufträge und WAP-Formulare	iot-gateway, sds-router
23	asset-management	10.0.20.32	8290	Geräte, Assets und Benutzerzuordnung	iot-gateway, subscriber-core, mobility-core, task-workflow
24	sip-switch	10.0.20.33	8300	Zentraler SIP Switch mit Serving-TBS-Routing und PBX-Fallback	iot-gateway, mobility-core
25	alert-service	10.0.20.34	8310	NINA/KATWARN, eigene Meldungen und GPS-gestützte Warn-SDS	control-room, sds-router

Die WebUIs liegen auf dem jeweiligen Managementport. /health/live und /health/ready dienen der Erstprüfung. Metrics-/OpenAPI-Verträge sind dienstabhängig: alert-service besitzt diese beiden Standardmarker im aktuellen Stand nicht und verlangt für die Fach-API ein Token.

2.2 Autorität und Zuständigkeit

- Die **TBS** bleibt Eigentümerin von PHY, MAC, LLC, MLE, lokaler MM/CMCE-Zeitkritik, lokaler Sprachführung und Air-PDU-Encoding.
- Die **Fachkerne** halten langlebige netzweite Zustände und Policies.

- Der **Control Room** zeigt Lage und Bedienwege, erzeugt aber keine zweite Wahrheit neben den Fachkernen.
- **Observability** überwacht, darf aber keinen Funk- oder Call-Pfad blockieren.
- shared/ ist eine Library und **kein** zusätzlicher Container.

2.3 Aktivierungsstrategie

Dienste mit shadow/authoritative werden zuerst im Shadow-Modus installiert. So lassen sich URLs, Abhängigkeiten und Zustandsmodelle prüfen, ohne sofort externe Nebenwirkungen, Kerneländerungen, OTAR oder Funkinjektionen auszulösen.

Dienst	Startmodus	Erst nach erfolgreichem Test auf authoritative
packet-core	shadow	packet.mode
ip-gateway	shadow	interface.mode
security-core	shadow	policy.operating_mode
kmf	shadow	policy.operating_mode
transit	shadow	region.operating_mode
application-gateway	shadow	runtime.operating_mode
media-library	shadow	runtime.operating_mode

3. Planung von Netzwerk, LXC und Storage

3.1 Managementnetz

Die Beispieltopologie verwendet 10.0.20.0/24. Sie kann ersetzt werden, muss aber in Inventory, gerenderten Konfigurationen, Firewallregeln und Basisstationskonfiguration konsistent bleiben. Ein Default-Gateway ist für den lokalen Corebetrieb nicht erforderlich; es wird nur für Paketupdates, externe Connectoren, Transit-WAN oder Internetzugang des IP Gateways benötigt.

Empfehlungen: - eigenes VLAN und eigene Firewallzone; - nur Deployment-/Adminhost, Basisstation und notwendige LXC dürfen zugreifen; - keine Portweiterleitung; - NTP intern bereitstellen; - statische DHCP-Leases oder feste Adressen; - Hostnamen aus deploy/open-lab/generated/hosts.example übernehmen oder internes DNS pflegen.

3.2 LXC-Ressourcen

Die folgenden Werte sind Labor-Empfehlungen, keine harten Mindestwerte. Da die Installer Rust lokal kompilieren, benötigen sie während des Builds mehr RAM und CPU als später im Runtimebetrieb. Bei 2 GB RAM sollte temporär zusätzlicher Swap vorhanden sein; komfortabler sind 4 GB während des Builds.

Dienst	vCPU	RAM	Disk	Hinweis
node-gateway	2	2 GB	8 GB	Standard-LXC
mobility-core	2	2 GB	8 GB	Standard-LXC
subscriber-core	2	2 GB	8 GB	Standard-LXC
group-core	2	2 GB	8 GB	Standard-LXC
call-control	2	2 GB	8 GB	Standard-LXC
media-switch	2-4	2-4 GB	12 GB	Standard-LXC
recorder	2	2 GB	16 GB + Archiv	Aufzeichnungen separat dimensionieren
sds-router	2	2 GB	8 GB	Standard-LXC
packet-core	2	2 GB	8 GB	Standard-LXC
ip-gateway	2	2 GB	12 GB + PCAP	/dev/net/tun; CAP_NET_ADMIN/RAW
security-core	2	2 GB	8 GB	restriktives Managementsegment
kmf	2	2 GB	12 GB	restriktives Managementsegment
transit	2	2 GB	8 GB	Standard-LXC
application-gateway	2	2 GB	12 GB	Standard-LXC
media-library	2-4	4 GB	32 GB + Archiv	Medien/Preview/Archive; ffmpeg
control-room	2	2 GB	12 GB	Standard-LXC
observability	4	4-8 GB	32-100 GB	Retention bestimmt Diskbedarf

3.3 Storage und NFS

- Live-State bleibt lokal auf dem LXC-Dateisystem.
- Recorder und Media Library dürfen Archive auf NFS ablegen, sollen aber nicht von einem langsamen NFS im zeitkritischen Pfad abhängen.
- KMF-Master-Key und KMF-Backup **nicht** am selben Ort sichern.
- Observability-Retention vorab an die Diskgröße anpassen.
- PCAP, Diagnosepakete und TAR-Exporte können sehr schnell wachsen.

Beispiel `/etc/fstab` für ein optionales Archiv:

```
10.0.20.5:/netcore-archive /mnt/nfs-share nfs4 rw,_netdev,nofail,x-systemd.automount,x-systemd.idle-  
timeout=60 0 0
```

Mit `nofail` und `Automount` blockiert ein fehlendes NAS den Start nicht unbegrenzt.

4. Installation der TETRA-Basisstation

4.1 Voraussetzungen

- Raspberry Pi 4/5 oder vergleichbarer 64-Bit-Linux-Rechner;
- Debian/Raspberry Pi OS 64 Bit mit systemd;
- funktionierendes SDR samt SoapySDR-Treiber;
- stabile Zeitbasis und ausreichend Kühlung;
- Netzwerkzugriff zum Node Gateway im Management-VLAN;
- optional NFS, ffmpeg, Piper, Asterisk/Brew.

4.2 Betriebssystempakete

```
sudo apt update
sudo apt install -y git curl ca-certificates build-essential pkg-config cmake clang \
  libsoapyhdr-dev soapyhdr-tools ffmpeg jq sqlite3 nfs-common
```

Gerätespezifische SoapySDR-Treiber zusätzlich installieren. Für eine komplett offline betriebene Station werden diese Pakete und der Rust-Toolchain vorab über ein internes Repository, Paketcache oder ein vorbereitetes Image bereitgestellt; zur Laufzeit benötigt die lokale TBS kein Internet.

Rust installieren:

```
curl --proto=https --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
rustup update stable
```

4.3 SDR prüfen

```
SoapySDRUtil --info
SoapySDRUtil --find
SoapySDRUtil --probe="driver=<TREIBER>"
```

Vor dem ersten Sendebetrieb Sample-Rate, Kanalnummer, Center-Frequenz, Gains und Passband prüfen. Dual-Carrier ist nur sinnvoll, wenn beide 25-kHz-Träger sauber im SDR-Passband liegen.

4.4 Benutzer und Verzeichnisse

```
sudo useradd --system --home /var/lib/netcore --shell /usr/sbin/nologin netcore 2>/dev/null || true
sudo install -d -o netcore -g netcore /var/lib/netcore /var/lib/netcore/recordings \
  /var/lib/netcore/audio /var/lib/netcore/tts/templates /var/cache/netcore/audio \
  /var/cache/netcore/tts /var/lib/flowstation
sudo install -d -m 0750 /etc/netcore /opt/netcore
```

Den Benutzer der Gerätegruppe des SDR hinzufügen, beispielsweise plugdev, dialout oder eine gerätespezifische Gruppe:

```
sudo usermod -aG plugdev,dialout netcore
```

4.5 Repository bereitstellen

Online per Git:

```
cd /opt
sudo git clone https://github.com/JanHG98/netcore-tetra.git netcore-tetra
sudo chown -R "$USER":"$USER" /opt/netcore-tetra
cd /opt/netcore-tetra
```

Den Release vor dem Build festhalten:

```
git fetch origin tag v1.9.0
git checkout --detach v1.9.0
```

Für Offline-Betrieb einen geprüften Quell- und Abhängigkeitsbestand dieses Releases vorbereiten.

4.6 Build und Installation

```
cd /opt/netcore-tetra
cargo build --locked --release -p bluestation-bs
sudo install -m 0755 target/release/bluestation-bs /usr/local/bin/bluestation-bs
sudo install -m 0640 -o root -g netcore Docs/basisstation.config.sanitized.example.toml \
  /etc/netcore/config.toml
sudo cp /etc/netcore/config.toml /etc/netcore/config.toml.fallback
sudo chmod 0640 /etc/netcore/config.toml /etc/netcore/config.toml.fallback
```

Die bereitgestellte bereinigte Konfiguration ist eine Startvorlage. Vor Start müssen Frequenzen, Netzwerkidentitäten, Dashboard-Zugang und Integrationen angepasst werden.

4.7 Erster manueller Start

```
sudo -u netcore RUST_LOG=info /usr/local/bin/bluestation-bs /etc/netcore/config.toml
```

Prüfen: TOML ohne Parserfehler, SDR erkannt, Downlink stabil, WebUI erreichbar, keine dauerhaften Underflow-/Passbandfehler. Mit Ctrl+C beenden.

4.8 systemd-Unit

[Unit]

```
Description=NetCore TETRA Basisstation
After=network-online.target
Wants=network-online.target
```

[Service]

```
Type=simple
User=netcore
Group=netcore
WorkingDirectory=/opt/netcore-tetra
ExecStart=/usr/local/bin/bluestation-bs /etc/netcore/config.toml
Restart=on-failure
RestartSec=5
TimeoutStopSec=20
LimitNOFILE=65536
Environment=RUST_LOG=info
```

[Install]

```
WantedBy=multi-user.target
```

Als /etc/systemd/system/tetra.service speichern:

```
sudo systemctl daemon-reload
sudo systemctl enable --now tetra.service
sudo systemctl status tetra.service --no-pager
sudo journalctl -u tetra.service -n 200 --no-pager
```

4.9 Optional: Piper TTS

```
sudo system-backend/tts/install-piper.sh
sudo systemctl status netcore-piper.service --no-pager
curl -fsS http://127.0.0.1:5005/voices | jq .
```

TTS erzeugt Dateien. Die eigentliche Funk-Aussendung erfolgt kontrolliert über Media Library/Recording-Workflow, nicht direkt aus der Synthese.

5. Konfiguration der Basisstation

5.1 Grundregeln

- Vor jeder Änderung `config.toml`, `config.toml.fallback` und letzte `.bak` sichern.
- Konfigurationsdateien nur für root und Dienstgruppe lesbar machen.
- Primärdatei nach Änderung manuell testen; erst danach systemd neu starten.
- Die Fallback-Datei konservativ halten und nicht automatisch überschreiben.
- Die mitgelieferte Originalkonfiguration enthielt standortspezifische Werte; der Guide liefert deshalb eine bereinigte Vorlage ohne ursprüngliche Zugangsdaten.

```
sudo cp /etc/netcore/config.toml /etc/netcore/config.toml.$(date +%F-%H%M).bak
sudo -u netcore /usr/local/bin/bluestation-bs /etc/netcore/config.toml
```

5.2 Pflichtsektionen

Sektion	Bedeutung
[phy_io] / [phy_io.soapysdr]	SDR-Backend, Frequenzen, Sample-Rate, Center-Frequenzen, Kanal und Gains.
[net_info]	MCC und MNC.
[cell_info]	Band, Carrier, Duplex, LAC, Colour Code, Dienste und Rufverhalten.
[dashboard]	Bind-Adresse, Port und lokaler Dashboard-Zugang.
[control_room]	Verbindung der TBS zum Node Gateway.
[edge_fallback]	Lokale Autonomie, Matrix-Lease, Polycache und Replay-Spool.

5.3 RF- und Zellparameter

Beispiel:

```
[phy_io]
backend = "SoapySdr"

[phy_io.soapysdr]
tx_freq = 418000000      # ANPASSEN
rx_freq = 408000000      # ANPASSEN
device = "driver=<TREIBER>"
sample_rate = 600000
tx_center_freq = 418012500    # bei Dual Carrier
rx_center_freq = 408012500

[net_info]
mcc = 1                  # ANPASSEN
mnc = 333                # ANPASSEN

[cell_info]
freq_band = 4
main_carrier = 720        # ANPASSEN
secondary_carrier = 721    # optional
duplex_spacing = 0
freq_offset = 0
reverse_operation = false
location_area = 1
colour_code = 1
timezone = "Europe/Berlin"
registration = true
deregistration = true
voice_service = true
sndcp_service = true
advanced_link = true
```

Carrier-Nummer und Center-Frequenz sind getrennte Ebenen: formal korrekte Carrier können trotzdem außerhalb des eingestellten SDR-Passbands liegen.

5.4 Packet Data auf der TBS

Die lokale [cell_info.wap_ip]- und [cell_info.packet_data_gateway]-Funktion bildet den Air-Interface-nahen Fallback. Beim zentralen Betrieb müssen TBS-Pool, Packet-Core-Pool und IP-Gateway-Netz zusammenpassen. Nicht gleichzeitig zwei Gateways mit derselben TETRA-IP aktiv routen lassen.

Empfohlener Übergang: 1. TBS Packet Data lokal testen. 2. Packet Core im shadow-Modus anbinden. 3. IP Gateway im shadow-Modus und Kernel-Plan prüfen. 4. In einem Wartungsfenster zentrale Autorität aktivieren. 5. Fallback der lokalen TBS getrennt testen.

5.5 SDS-Kommandos

[cell_info.sds_command_control] darf nur explizit autorisierte ISSI enthalten. Restart/Shutdown sind RF-wirksame Aktionen. Im offenen Labornetz darf der Managementzugang nicht als Ersatz für diese Allowlist missverstanden werden.

5.6 Dashboard, Recording, Audio und TTS

- Dashboard-Passwort sofort ändern; Port nur im Managementnetz öffnen.
- Recording- und Audioverzeichnisse auf Eigentümer/Rechte prüfen.
- NFS-Archive mit nofail/Automount anbinden.
- TTS-Endpunkt standardmäßig lokal `http://127.0.0.1:5005` oder über Application Gateway/Media Library verwenden.
- Ohne echten TETRA-Encoder bleiben WAV/MP3 nur previewfähig.

5.7 Node-Gateway-Anbindung

```
[control_room]
enabled = true
host = "10.0.20.10"
port = 8080
use_tls = false
endpoint_path = "/ws/node"
node_id = "SRV-M-TBS-01"
station_name = "SRV-M-TBS-01"
site = "Main"
central_sds_routing = false
```

Die TBS verbindet sich in der verteilten LXC-Topologie **zum Node Gateway**, nicht direkt zum Control Room. `central_sds_routing` erst aktivieren, wenn SDS Router, Node Gateway und Fallback getestet sind.

5.8 Edge-Fallback

```
[edge_fallback]
enabled = true
enter_after_secs = 15
recover_after_secs = 20
unknown_service_is_available = false
service_matrix_lease_secs = 60
policy_cache_path = "/var/lib/flowstation/edge-policy-cache.json"
policy_cache_max_age_secs = 604800
keep_last_known_policy = true
event_spool_path = "/var/lib/flowstation/edge-event-spool.jsonl"
event_spool_max_entries = 10000
event_spool_max_bytes = 16777216
replay_batch_size = 128
required_services = ["subscriber-core", "group-core", "mobility-core", "call-control", "media-switch", "sds-router"]
```

`unknown_service_is_available=false` ist fail-closed. Eine veraltete oder unbekannte zentrale Dienstlage wird nicht als gesund angenommen. `keep_last_known_policy=true` verhindert, dass eine isolierte TBS unbemerkt von einer restriktiven Policy in ein offenes Netz fällt.

5.9 Primär- und Fallback-Konfiguration

Kann die Primärdatei nicht geladen werden, versucht die TBS `<datei>.fallback`. Das Dashboard zeigt den Fallbackbetrieb dauerhaft an. Nach einem solchen Start:

```
sudo journalctl -u tetra.service -b --no-pager | grep -iE 'config|fallback|parse|error'
diff -u /etc/netcore/config.toml.fallback /etc/netcore/config.toml
```

6. Proxmox-LXC vorbereiten

6.1 Gemeinsame LXC-Basis

Empfohlen: Debian 13, unprivilegierter LXC, statische IP, nesting=1, 2 Kerne, 2 GB RAM, 512 MB Swap, Autostart. Beispiel aus dem Repository:

```
unprivileged: 1
features: nesting=1
memory: 2048
swap: 512
cores: 2
onboot: 1
startup: order=20,up=20,down=30
```

6.2 Pakete pro LXC

```
apt update
apt install -y ca-certificates curl git openssh-server build-essential pkg-config cmake clang jq
curl --proto =https --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
source /root/.cargo/env
rustup update stable
systemctl enable --now ssh
```

Media Library benötigt zusätzlich ffmpeg; Recorder/Media Library optional nfs-common; IP Gateway zusätzlich iproute2 nftables tcpdump dnsutils.

6.3 SSH vom Deployment-Host

```
ssh-keygen -t ed25519 -f ~/.ssh/netcore-deploy
ssh-copy-id -i ~/.ssh/netcore-deploy.pub root@10.0.20.10
# für alle weiteren LXC wiederholen
```

Inventory-ssh_options gegebenenfalls um -i ~/.ssh/netcore-deploy ergänzen.

6.4 IP Gateway - TUN-Passthrough

Auf dem Proxmox-Host in /etc/pve/lxc/<CTID>.conf:

```
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net/tun dev/net/tun none bind,create=file
```

Danach Container neu starten und prüfen:

```
ls -l /dev/net/tun
ip tuntap add dev ntc-test mode tun
ip link del ntc-test
```

6.5 Recorder/Media Library NFS

NFS kann im LXC selbst gemountet oder als Proxmox-Mountpoint durchgereicht werden. Schreibrechte müssen zum jeweiligen Dienstbenutzer passen. Live-State bleibt lokal; nur Archivdaten gehen auf NFS.

7. Automatisches Deployment aller LXC

7.1 Korrigiertes Inventory verwenden

Im Repository-Inventory zeigt `control-room.config_target` auf `/etc/netcore/control-room.toml`, während Installer und systemd-Unit `/etc/netcore-control-room/control-room.toml` verwenden. Der mit diesem Guide gelieferte Inventory-Entwurf korrigiert diesen Pfad. Ohne Korrektur würde der Deployer eine Datei schreiben, die der Dienst nicht liest.

```
cp deploy/open-lab/inventory.example.toml deploy/open-lab/inventory.toml
${EDITOR:-nano} deploy/open-lab/inventory.toml
```

Anpassen: Hosts, SSH-Benutzer/Optionen, ggf. Ports, Remote-Quellpfad und Serviceauswahl.

7.2 Validieren und rendern

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml plan
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml render
```

`render` ersetzt Dienst-URLs passend zum Inventory und erzeugt Servicekatalog, Hosts-Datei, Portliste und Abhängigkeitsgraph. Vor dem Deploy die gerenderten Configs auf falsche Loopback-Adressen, Netzbereiche und Betriebsmodi prüfen.

7.3 Dry Run und Deployment

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply --dry-run
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml status
```

Der Deployer erstellt ein deterministisches Quellbundle ohne PDFs, `.git`, `target`, `Caches` oder `Node-Module`, kopiert es auf jedes Ziel, ruft den dienstspezifischen Installer auf, kopiert die gerenderte Konfiguration und startet in Abhängigkeitsreihenfolge.

7.4 Selektive Updates

Ein einzelner Dienst kann zusammen mit seinen transitiven Abhängigkeiten ausgewählt werden. Vor einem Update immer State-Backup des Dienstes und eine Kopie der aktiven Konfiguration anlegen. Das Deployment überschreibt keine Fach-State-Dateien, kann aber Binärdatei, Unit und Konfiguration aktualisieren.

7.5 Erstprüfung nach Deployment

```
for hp in 10.0.20.10:8080 10.0.20.11:8090 10.0.20.12:8100 10.0.20.13:8110 \
10.0.20.14:8120 10.0.20.15:8130 10.0.20.16:8140 10.0.20.17:8150 \
10.0.20.18:8160 10.0.20.19:8170 10.0.20.20:8180 10.0.20.21:8190 \
10.0.20.22:8200 10.0.20.23:8220 10.0.20.24:8230 10.0.20.25:9010 \
10.0.20.26:8210; do
curl -fsS "http://$hp/health/live" >/dev/null && echo "OK $hp" || echo "FAIL $hp"
done
```

8. Manuelle Installation und Bedienung je Dienst

8.1 Gemeinsames Muster

Auf dem jeweiligen LXC:

```
cd /opt/netcore-tetra
sudo system-backend/<dienst>/install/install.sh
sudo editor <config-pfad>
sudo systemctl restart <unit>
sudo systemctl status <unit> --no-pager
curl -fsS http://127.0.0.1:<port>/health/live
curl -i http://127.0.0.1:<port>/health/ready
```

Readiness kann beim Start oder bei fehlender Abhängigkeit nachvollziehbar 503 liefern; Liveness muss bei laufendem Prozess 200 liefern.

8.2 node-gateway

Zweck: Zentraler Einstiegspunkt für TBS-WebSockets und normalisierter Transport zu den Backend-Diensten.

Eigenschaft	Wert
WebUI/API	http://10.0.20.10:8080/
systemd	netcore-node-gateway.service
Konfiguration	/etc/netcore/node-gateway.toml
Installer	system-backend/node-gateway/install/install.sh
Abhängigkeiten	keine
State	In-Memory; keine Fachdatenbank

Vor dem ersten Start: - service_monitor.targets müssen auf alle 16 anderen LXCs zeigen - Bind-Adresse und WebSocket-Pfade unverändert lassen, sofern kein Reverse Proxy eingesetzt wird

Bedienung in der WebUI: - TBS-Nodes und Heartbeats prüfen - Node-Ping auslösen - stale oder doppelte Sessions trennen - Core-Service-Matrix und Ereignisse kontrollieren

Prüfbefehle:

```
systemctl status netcore-node-gateway.service --no-pager
journalctl -u netcore-node-gateway.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8080/health/live
curl -i http://127.0.0.1:8080/health/ready
curl -fsS http://127.0.0.1:8080/api/v1/status | jq .
```

8.3 mobility-core

Zweck: Teilnehmerlage, Serving-Node-Zuordnung und MM-Context-Transfers zwischen TBS.

Eigenschaft	Wert
WebUI/API	http://10.0.20.11:8090/
systemd	netcore-mobility-core.service
Konfiguration	/etc/netcore/mobility-core.toml
Installer	system-backend/mobility-core/install/install.sh
Abhängigkeiten	node-gateway
State	aktuell primär Laufzeitlage

Vor dem ersten Start: - node_gateway.url auf ws://:8080/ws/backend setzen

Bedienung in der WebUI: - Serving Node je ISSI prüfen - Context Transfer starten und Phasen beobachten - Transfers kontrolliert abbrechen - RSSI/Energy-Saving-Lage prüfen

Prüfbefehle:

```
systemctl status netcore-mobility-core.service --no-pager
journalctl -u netcore-mobility-core.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8090/health/live
curl -i http://127.0.0.1:8090/health/ready
curl -fsS http://127.0.0.1:8090/api/v1/status | jq .
```

8.4 subscriber-core

Zweck: Zentrale Teilnehmerprofile und Admission-Policy.

Eigenschaft	Wert
WebUI/API	http://10.0.20.12:8100/
systemd	netcore-subscriber-core.service
Konfiguration	/etc/netcore/subscriber-core.toml
Installer	system-backend/subscriber-core/install/install.sh
Abhängigkeiten	node-gateway
State	/var/lib/netcore-subscriber-core/subscribers.json

Vor dem ersten Start: - access_policy.mode bewusst wählen - bei allow_list zuerst mindestens einen Admin-/Testteilnehmer anlegen - node_gateway.url setzen

Bedienung in der WebUI: - Teilnehmer anlegen/ändern/sperrern - Allowlist oder Open Network wählen - Import/Export durchführen - TBS-Synchronisation prüfen

Prüfbefehle:

```
systemctl status netcore-subscriber-core.service --no-pager
journalctl -u netcore-subscriber-core.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8100/health/live
curl -i http://127.0.0.1:8100/health/ready
curl -fsS http://127.0.0.1:8100/api/v1/status | jq .
```

8.5 group-core

Zweck: GSSI-Stammdaten, Mitgliedschaften, Affiliationen und DGNA.

Eigenschaft	Wert
WebUI/API	http://10.0.20.13:8110/
systemd	netcore-group-core.service
Konfiguration	/etc/netcore/group-core.toml
Installer	system-backend/group-core/install/install.sh
Abhängigkeiten	node-gateway, subscriber-core
State	/var/lib/netcore-group-core/groups.json

Vor dem ersten Start: - allow_unlisted_groups=false ist der sichere Ausgangspunkt - node_gateway.url setzen

Bedienung in der WebUI: - Gruppenprofile anlegen - Mitgliedschaften pflegen - Affiliationen beobachten - DGNA Attach/Detach auslösen

Prüfbefehle:

```
systemctl status netcore-group-core.service --no-pager
journalctl -u netcore-group-core.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8110/health/live
curl -i http://127.0.0.1:8110/health/ready
curl -fsS http://127.0.0.1:8110/api/v1/status | jq .
```

8.6 call-control

Zweck: Netzweite logische Gruppen-/Individualrufe, Call Legs, Floor und Restore.

Eigenschaft	Wert
WebUI/API	http://10.0.20.14:8120/
systemd	netcore-call-control.service
Konfiguration	/etc/netcore/call-control.toml
Installer	system-backend/call-control/install/ install.sh
Abhängigkeiten	node-gateway, subscriber-core, group-core, mobility-core
State	/var/lib/netcore-call-control/calls.json

Vor dem ersten Start: - node_gateway.url setzen - Ruf- und Restore-Timeouts an Labornetz anpassen

Bedienung in der WebUI: - Rufe starten/beenden - Floor Holder und Queue beobachten - Operator-Floor nur gezielt erzwingen - Restore-Vorgänge prüfen

Prüfbefehle:

```
systemctl status netcore-call-control.service --no-pager
journalctl -u netcore-call-control.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8120/health/live
curl -i http://127.0.0.1:8120/health/ready
curl -fsS http://127.0.0.1:8120/api/v1/status | jq .
```

8.7 media-switch

Zweck: Routing bereits codierter 35-Byte-TETRA-Sprachframes zwischen TBS-Call-Legs.

Eigenschaft	Wert
WebUI/API	http://10.0.20.15:8130/
systemd	netcore-media-switch.service
Konfiguration	/etc/netcore/media-switch.toml
Installer	system-backend/media-switch/install/ install.sh
Abhängigkeiten	node-gateway, call-control
State	zeitkritische Laufzeitdaten im Speicher

Vor dem ersten Start: - node_gateway.url und call_control.url setzen - Recorder-Tap-Historie passend dimensionieren

Bedienung in der WebUI: - Sessions und Jitter-Puffer beobachten - Streams stummschalten - Puffer nur zur Fehlersuche leeren - Testframe-Injection ausschließlich im Labor nutzen

Prüfbefehle:

```
systemctl status netcore-media-switch.service --no-pager
journalctl -u netcore-media-switch.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8130/health/live
curl -i http://127.0.0.1:8130/health/ready
curl -fsS http://127.0.0.1:8130/api/v1/status | jq .
```

8.8 recorder

Zweck: Passive, verlustfreie Aufzeichnung von TACELP-Frames außerhalb des Rufpfads.

Eigenschaft	Wert
WebUI/API	http://10.0.20.16:8140/
systemd	netcore-recorder.service

Eigenschaft	Wert
Konfiguration	/etc/netcore/recorder.toml
Installer	system-backend/recorder/install/install.sh
Abhängigkeiten	media-switch
State	/var/lib/netcore-recorder/recordings

Vor dem ersten Start: - media_switch.tap_url/sessions_url setzen - Storage und freien Speicher prüfen - NFS nur als separates Archiv verwenden

Bedienung in der WebUI: - aktive Aufnahmen beobachten - Integrität prüfen - Retention/Legal Hold setzen - TAR exportieren oder Aufnahme löschen

Prüfbefehle:

```
systemctl status netcore-recorder.service --no-pager
journalctl -u netcore-recorder.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8140/health/live
curl -i http://127.0.0.1:8140/health/ready
curl -fsS http://127.0.0.1:8140/api/v1/status | jq .
```

8.9 sds-router

Zweck: Zentrale SDS-/Statusvermittlung, Store-and-forward und Anwendungsrouten.

Eigenschaft	Wert
WebUI/API	http://10.0.20.17:8150/
systemd	netcore-sds-router.service
Konfiguration	/etc/netcore/sds-router.toml
Installer	system-backend/sds-router/install/install.sh
Abhängigkeiten	node-gateway, subscriber-core, group-core, mobility-core
State	/var/lib/netcore-sds-router/messages.json

Vor dem ersten Start: - node_gateway.url setzen - TBS central_sds_routing erst nach Test aktivieren - TTL und Payloadgrenzen prüfen

Bedienung in der WebUI: - Nachrichten senden/suchen - Retry/Requeue/Cancel durchführen - Routen verwalten - Application-Outbox quittieren

Prüfbefehle:

```
systemctl status netcore-sds-router.service --no-pager
journalctl -u netcore-sds-router.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8150/health/live
curl -i http://127.0.0.1:8150/health/ready
curl -fsS http://127.0.0.1:8150/api/v1/status | jq .
```

8.10 packet-core

Zweck: PDP-/NSAPI-State-Machine, IPv4-Leases, Mobility Anchoring, Fragmente und Flow Control.

Eigenschaft	Wert
WebUI/API	http://10.0.20.18:8160/
systemd	netcore-packet-core.service
Konfiguration	/etc/netcore/packet-core.toml
Installer	system-backend/packet-core/install/install.sh
Abhängigkeiten	node-gateway, subscriber-core, mobility-core

Eigenschaft	Wert
State	/var/lib/netcore-packet-core/state.json

Vor dem ersten Start: - zunächst packet.mode=shadow - node_gateway.url setzen - Adresspool muss zum IP Gateway passen

Bedienung in der WebUI: - Kontexte/NSAPI prüfen - Wake/End-of-Data/Modify/Deactivate auslösen - Bearers und Reassemblies beobachten - Downlink-N-PDUs kontrollieren

Prüfbefehle:

```
systemctl status netcore-packet-core.service --no-pager
journalctl -u netcore-packet-core.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8160/health/live
curl -i http://127.0.0.1:8160/health/ready
curl -fsS http://127.0.0.1:8160/api/v1/status | jq .
```

8.11 ip-gateway

Zweck: Layer-3-Übergang zwischen Packet Core und normalen IPv4-Netzen.

Eigenschaft	Wert
WebUI/API	http://10.0.20.19:8170/
systemd	netcore-ip-gateway.service
Konfiguration	/etc/netcore/ip-gateway.toml
Installer	system-backend/ip-gateway/install/install.sh
Abhängigkeiten	packet-core
State	/var/lib/netcore-ip-gateway/state.json und captures/

Vor dem ersten Start: - /dev/net/tun im LXC durchreichen - zunächst interface.mode=shadow - packet_core.url und TETRA-IP-Netz konsistent setzen - vor authoritative nftables-Regeln prüfen

Bedienung in der WebUI: - Kernel-Plan prüfen - Routen/NAT/Firewall verwalten - DNS/Testserver nutzen - PCAP-Captures starten und stoppen

Prüfbefehle:

```
systemctl status netcore-ip-gateway.service --no-pager
journalctl -u netcore-ip-gateway.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8170/health/live
curl -i http://127.0.0.1:8170/health/ready
curl -fsS http://127.0.0.1:8170/api/v1/status | jq .
```

Authoritative-Check: Vor Umschaltung GET /api/v1/kernel/plan prüfen. Danach TUN, Routen und nftables separat mit ip addr, ip route und nft list ruleset kontrollieren.

8.12 security-core

Zweck: Security-Class-Policy, Lab-Authentisierung, DCK-Kontexte, Sperren und Audit.

Eigenschaft	Wert
WebUI/API	http://10.0.20.20:8180/
systemd	netcore-security-core.service
Konfiguration	/etc/netcore/security-core.toml
Installer	system-backend/security-core/install/install.sh
Abhängigkeiten	node-gateway, subscriber-core
State	/var/lib/netcore-security-core/state.json; Seed separat

Vor dem ersten Start: - zunächst policy.operating_mode=shadow - Lab-Provider nicht als produktive TETRA-Kryptografie betrachten - kein stilles Downgrade konfigurieren

Bedienung in der WebUI: - Profile und Security Class verwalten - Challenges/Alarmer beobachten - Teilnehmer/Geräte sperren oder freigeben - Edge-Aktionen quittieren

Prüfbefehle:

```
systemctl status netcore-security-core.service --no-pager
journalctl -u netcore-security-core.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8180/health/live
curl -i http://127.0.0.1:8180/health/ready
curl -fsS http://127.0.0.1:8180/api/v1/status | jq .
```

8.13 kmf

Zweck: Lifecycle für CCK/GCK/SCK, Rotation, Crypto Periods und OTAR-Orchestrierung.

Eigenschaft	Wert
WebUI/API	http://10.0.20.21:8190/
systemd	netcore-kmf.service
Konfiguration	/etc/netcore/kmf.toml
Installer	system-backend/kmf/install/install.sh
Abhängigkeiten	security-core
State	/var/lib/netcore-kmf/state.json + vault.json + master.key

Vor dem ersten Start: - zunächst policy.operating_mode=shadow - Master-Key separat sichern - Managementnetz besonders restriktiv halten

Bedienung in der WebUI: - Keys erzeugen/rotieren/aktivieren/widerrufen - Vier-Augen-Jobs freigeben - OTAR-Zustellungen und ACKs prüfen - Backups erzeugen

Prüfbefehle:

```
systemctl status netcore-kmf.service --no-pager
journalctl -u netcore-kmf.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8190/health/live
curl -i http://127.0.0.1:8190/health/ready
curl -fsS http://127.0.0.1:8190/api/v1/status | jq .
```

Backup-Hinweis: master.key getrennt vom normalen KMF-Backup sichern. Ohne Master-Key ist ein Vault-Backup nicht nutzbar; zusammen am selben Ort wäre die Trennung wirkungslos.

8.14 transit

Zweck: NetCore-native Regionalvermittlung mit Peers, Routen, Sessions und Failover.

Eigenschaft	Wert
WebUI/API	http://10.0.20.22:8200/
systemd	netcore-transit.service
Konfiguration	/etc/netcore/transit.toml
Installer	system-backend/transit/install/install.sh
Abhängigkeiten	mobility-core, call-control, media-switch, sds-router
State	/var/lib/netcore-transit/state.json

Vor dem ersten Start: - region_id/swmi_id/advertised_endpoint eindeutig setzen - zunächst operating_mode=shadow - kein ETSI-ISI behaupten

Bedienung in der WebUI: - Regionen/Peers verwalten - Routen und Erreichbarkeit prüfen - Sessions/Queues beobachten - Failover kontrolliert auslösen

Prüfbefehle:

```
systemctl status netcore-transit.service --no-pager
journalctl -u netcore-transit.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8200/health/live
curl -i http://127.0.0.1:8200/health/ready
curl -fsS http://127.0.0.1:8200/api/v1/status | jq .
```

8.15 application-gateway

Zweck: Adapter- und Workflow-Gateway für externe Anwendungen, Webhooks, Vorlagen und TTS.

Eigenschaft	Wert
WebUI/API	http://10.0.20.23:8220/
systemd	netcore-application-gateway.service
Konfiguration	/etc/netcore/application-gateway.toml
Installer	system-backend/application-gateway/install/install.sh
Abhängigkeiten	sds-router
State	state.json, secrets.json, spool, backups

Vor dem ersten Start: - interne URLs rendern - zunächst runtime.operating_mode=shadow - Secrets ausschließlich über secrets.json/WebUI pflegen

Bedienung in der WebUI: - Connectoren testen - Routen/Vorlagen pflegen - manuelle Dispatches auslösen - TTS-Jobs publizieren - Dead Letters erneut zustellen

Prüfbefehle:

```
systemctl status netcore-application-gateway.service --no-pager
journalctl -u netcore-application-gateway.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8220/health/live
curl -i http://127.0.0.1:8220/health/ready
curl -fsS http://127.0.0.1:8220/api/v1/status | jq .
```

8.16 media-library

Zweck: Zentrale Medienablage, Vorschau, Freigabe, TACELP-Cache und Playout.

Eigenschaft	Wert
WebUI/API	http://10.0.20.24:8230/
systemd	netcore-media-library.service
Konfiguration	/etc/netcore/media-library.toml
Installer	system-backend/media-library/install/install.sh
Abhängigkeiten	media-switch, recorder, application-gateway
State	/var/lib/netcore-media-library/assets + state.json

Vor dem ersten Start: - Abhängigkeits-URLs setzen - ffmpeg installieren - zunächst runtime.operating_mode=shadow - TACELP-Encoder/Decoder bei Bedarf konfigurieren

Bedienung in der WebUI: - Assets hochladen/importieren - Vorschau prüfen - Assets freigeben/ablehnen - Playout-Jobs in bestehende Sessions starten - Archivkopien prüfen

Prüfbefehle:

```
systemctl status netcore-media-library.service --no-pager
journalctl -u netcore-media-library.service -n 200 --no-pager
curl -fsS http://127.0.0.1:8230/health/live
curl -i http://127.0.0.1:8230/health/ready
curl -fsS http://127.0.0.1:8230/api/v1/status | jq .
```

8.17 control-room

Zweck: Zentrale Lage-, Operator-, Incident- und Schichtbuchebebene.

Eigenschaft	Wert
WebUI/API	http://10.0.20.25:9010/
systemd	netcore-control-room.service
Konfiguration	/etc/netcore-control-room/control-room.toml
Installer	system-backend/control-room/install/install.sh
Abhängigkeiten	node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library
State	control-room.sqlite3 + operations.json

Vor dem ersten Start: - alle Service-base_urls setzen - Config-Pfad /etc/netcore-control-room/control-room.toml verwenden - Open-Lab-Rechte bedenken

Bedienung in der WebUI: - Gesamtlage und kritische Dienste prüfen - Incidents quittieren/lösen - Schichtbuch führen - Schnellaktionen Kick/Clear Emergency/DGNA verwenden - Fach-WebUIs öffnen

Prüfbefehle:

```
systemctl status netcore-control-room.service --no-pager
journalctl -u netcore-control-room.service -n 200 --no-pager
curl -fsS http://127.0.0.1:9010/health/live
curl -i http://127.0.0.1:9010/health/ready
curl -fsS http://127.0.0.1:9010/api/v1/status | jq .
```

Pfadhinweis: Dieser Dienst ist die Ausnahme: aktive Konfiguration liegt unter /etc/netcore-control-room/control-room.toml.

8.18 observability

Zweck: Zentrale Metriken, Logs, Traces, Alerts, Silences und Diagnosepakete.

Eigenschaft	Wert
WebUI/API	http://10.0.20.26:8210/
systemd	netcore-observability.service
Konfiguration	/etc/netcore/observability.toml
Installer	system-backend/observability/install/install.sh
Abhängigkeiten	node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library, control-room
State	/var/lib/netcore-observability/state.json + diagnostics/

Vor dem ersten Start: - alle Targets korrekt rendern - Retention an Disk anpassen - klassischen Stack nur installieren, wenn Binaries vorhanden sind

Bedienung in der WebUI: - Scrape Targets testen - Metrikserien und Logs durchsuchen - Alarme quittieren - Silences setzen - Diagnosepakete erstellen

Prüfbefehle:

```
systemctl status netcore-observability.service --no-pager
journalctl -u netcore-observability.service -n 200 --no-pager
```

```
curl -fsS http://127.0.0.1:8210/health/live
curl -i http://127.0.0.1:8210/health/ready
curl -fsS http://127.0.0.1:8210/api/v1/status | jq .
```

8.19 iot-gateway

MQTT, Home Assistant, Homematic und Command/Ack-Ledger.

Merkmal	Wert
Beispielhost	10.0.20.27
Port	8240
Unit	netcore-iot-gateway.service
Installer	system-backend/iot-gateway/install/install.sh
Konfigurationsvorlage	system-backend/iot-gateway/config/iot-gateway.example.toml
Konfigurationsziel	/etc/netcore/iot-gateway.toml
Abhängigkeiten	node-gateway, mobility-core, call-control, sds-router

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

Reale Akteur-/Integrationsaktionen bleiben bis zur bewussten Abnahme gesperrt; Command-ID, Ack und tatsächliche Zielwirkung zusammen prüfen.

8.20 hardware-gateway

Hardware-I/O, Sensoren und begrenzte Akteurbefehle.

Merkmal	Wert
Beispielhost	10.0.20.28
Port	8250
Unit	netcore-hardware-gateway.service
Installer	system-backend/hardware-gateway/install/install.sh
Konfigurationsvorlage	system-backend/hardware-gateway/config/hardware-gateway.example.toml
Konfigurationsziel	/etc/netcore/hardware-gateway.toml
Abhängigkeiten	iot-gateway

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

Reale Akteur-/Integrationsaktionen bleiben bis zur bewussten Abnahme gesperrt; Command-ID, Ack und tatsächliche Zielwirkung zusammen prüfen.

8.21 rf-monitor

RF-Messwerte, Qualität und Diagnose.

Merkmal	Wert
Beispielhost	10.0.20.29
Port	8260
Unit	netcore-rf-monitor.service
Installer	system-backend/rf-monitor/install/install.sh
Konfigurationsvorlage	system-backend/rf-monitor/config/rf-monitor.example.toml

Merkmal	Wert
Konfigurationsziel	/etc/netcore/rf-monitor.toml
Abhängigkeiten	iot-gateway

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

8.22 alarm-workflow

Alarmregeln, Ereignisse und Eskalation.

Merkmal	Wert
Beispielhost	10.0.20.30
Port	8270
Unit	netcore-alarm-workflow.service
Installer	system-backend/alarm-workflow/install/install.sh
Konfigurationsvorlage	system-backend/alarm-workflow/config/alarm-workflow.example.toml
Konfigurationsziel	/etc/netcore/alarm-workflow.toml
Abhängigkeiten	iot-gateway, sds-router, hardware-gateway, rf-monitor

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

Reale Akteur-/Integrationsaktionen bleiben bis zur bewussten Abnahme gesperrt; Command-ID, Ack und tatsächliche Zielwirkung zusammen prüfen.

8.23 task-workflow

Strukturierte Aufträge und WAP-Formulare.

Merkmal	Wert
Beispielhost	10.0.20.31
Port	8280
Unit	netcore-task-workflow.service
Installer	system-backend/task-workflow/install/install.sh
Konfigurationsvorlage	system-backend/task-workflow/config/task-workflow.example.toml
Konfigurationsziel	/etc/netcore/task-workflow.toml
Abhängigkeiten	iot-gateway, sds-router

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

8.24 asset-management

Geräte, Assets und Benutzerzuordnung.

Merkmal	Wert
Beispielhost	10.0.20.32
Port	8290
Unit	netcore-asset-management.service

Merkmal	Wert
Installer	system-backend/asset-management/install/install.sh
Konfigurationsvorlage	system-backend/asset-management/config/asset-management.example.toml
Konfigurationsziel	/etc/netcore/asset-management.toml
Abhängigkeiten	iot-gateway, subscriber-core, mobility-core, task-workflow

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

8.25 sip-switch

Zentraler SIP Switch mit Serving-TBS-Routing und PBX-Fallback.

Merkmal	Wert
Beispielhost	10.0.20.33
Port	8300
Unit	netcore-sip-switch.service
Installer	system-backend/sip-switch/install/install.sh
Konfigurationsvorlage	system-backend/sip-switch/config/sip-switch.example.toml
Konfigurationsziel	/etc/netcore/sip-switch.toml
Abhängigkeiten	iot-gateway, mobility-core

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

8.26 alert-service

NINA/KATWARN, eigene Meldungen und GPS-gestützte Warn-SDS.

Merkmal	Wert
Beispielhost	10.0.20.34
Port	8310
Unit	netcore-alert-service.service
Installer	system-backend/alert-service/install/install.sh
Konfigurationsvorlage	system-backend/alert-service/config/alert-service.example.toml
Konfigurationsziel	/etc/netcore/alert-service.toml
Abhängigkeiten	control-room, sds-router

Vor Ausführung des Installers die Standortkonfiguration und Persistenz sichern. Nach dem Start Unit, live/ready und eine fachliche Testtransaktion prüfen.

Token, Versandfreigabe, SQLite-Sicherung und Inbetriebnahmereihenfolge stehen vollständig in Anhang B.

9. Konfigurationsreferenz der LXC-Dienste

Diese Referenz nennt die aktuellen Vorlagen für alle 25 Dienste. Die vollständigen Werte stehen in den verlinkten, auf den Release fixierten TOML-Dateien. Standortwerte aus dem Inventory und der aktiven Unit haben Vorrang vor historischen Beispieladressen. Schlüssel dürfen nur mit dem passenden Dienstbuild übernommen werden.

9.1 node-gateway

Quellvorlage · Ziel `/etc/netcore/node-gateway.toml` · Unit `netcore-node-gateway.service`.

Schlüssel	Beispielwert
<code>server.bind</code>	<code>"0.0.0.0:8080"</code>
<code>server.node_path</code>	<code>"/ws/node"</code>
<code>server.backend_path</code>	<code>"/ws/backend"</code>
<code>server.history_limit</code>	<code>1000</code>
<code>server.stale_after_secs</code>	<code>20</code>
<code>server.hello_timeout_secs</code>	<code>10</code>
<code>server.application_ping_secs</code>	<code>15</code>
<code>security.mode</code>	<code>"open_lab"</code>
<code>security.allow_remote_management</code>	<code>true</code>
<code>limits.max_message_bytes</code>	<code>1048576</code>
<code>limits.max_http_body_bytes</code>	<code>1048576</code>
<code>service_monitor.enabled</code>	<code>true</code>
<code>service_monitor.interval_secs</code>	<code>5</code>
<code>service_monitor.timeout_ms</code>	<code>1500</code>
<code>service_monitor.failure_threshold</code>	<code>2</code>
<code>service_monitor.recovery_threshold</code>	<code>2</code>
<code>service_monitor.targets[0].name</code>	<code>"mobility-core"</code>
<code>service_monitor.targets[0].url</code>	<code>"http://10.0.20.11:8090/health/ready"</code>
<code>service_monitor.targets[0].critical_for_edge</code>	<code>true</code>
<code>service_monitor.targets[0].fallback_mode</code>	<code>"local_registration_and_location_area"</code>
<code>service_monitor.targets[1].name</code>	<code>"subscriber-core"</code>
<code>service_monitor.targets[1].url</code>	<code>"http://10.0.20.12:8100/health/ready"</code>
<code>service_monitor.targets[1].critical_for_edge</code>	<code>true</code>
<code>service_monitor.targets[1].fallback_mode</code>	<code>"cached_policy_then_static_config"</code>
<code>service_monitor.targets[2].name</code>	<code>"group-core"</code>
<code>service_monitor.targets[2].url</code>	<code>"http://10.0.20.13:8110/health/ready"</code>

Schlüssel	Beispielwert
service_monitor.targets[2].critical_for_edge	true
service_monitor.targets[2].fallback_mode	"cached_policy_then_local_affiliations"
service_monitor.targets[3].name	"call-control"
service_monitor.targets[3].url	"http://10.0.20.14:8120/health/ready"
service_monitor.targets[3].critical_for_edge	true
service_monitor.targets[3].fallback_mode	"local_cell_calls_only"
service_monitor.targets[4].name	"media-switch"
service_monitor.targets[4].url	"http://10.0.20.15:8130/health/ready"
service_monitor.targets[4].critical_for_edge	true
service_monitor.targets[4].fallback_mode	"local_air_interface_media_only"
service_monitor.targets[5].name	"recorder"
service_monitor.targets[5].url	"http://10.0.20.16:8140/health/ready"
service_monitor.targets[5].critical_for_edge	false
service_monitor.targets[5].fallback_mode	"local_recorder_continues"
service_monitor.targets[6].name	"sds-router"
service_monitor.targets[6].url	"http://10.0.20.17:8150/health/ready"
service_monitor.targets[6].critical_for_edge	true
service_monitor.targets[6].fallback_mode	"local_delivery_and_durable_store_forward"
service_monitor.targets[7].name	"packet-core"
service_monitor.targets[7].url	"http://10.0.20.18:8160/health/ready"
service_monitor.targets[7].critical_for_edge	false
service_monitor.targets[7].fallback_mode	"local_sndcp_contexts"
service_monitor.targets[8].name	"ip-gateway"
service_monitor.targets[8].url	"http://10.0.20.19:8170/health/ready"
service_monitor.targets[8].critical_for_edge	false
service_monitor.targets[8].fallback_mode	"local_tun_gateway_when_configured"
service_monitor.targets[9].name	"security-core"
service_monitor.targets[9].url	"http://10.0.20.20:8180/health/ready"

Schlüssel	Beispielwert
service_monitor.targets[9].critical_for_edge	false
service_monitor.targets[9].fallback_mode	"last_known_security_policy_no_downgrade"
service_monitor.targets[10].name	"kmf"
service_monitor.targets[10].url	"http://10.0.20.21:8190/health/ready"
service_monitor.targets[10].critical_for_edge	false
service_monitor.targets[10].fallback_mode	"installed_keys_only_no_otar"
service_monitor.targets[11].name	"transit"
service_monitor.targets[11].url	"http://10.0.20.22:8200/health/ready"
service_monitor.targets[11].critical_for_edge	false
service_monitor.targets[11].fallback_mode	"no_inter_region_routing"
service_monitor.targets[12].name	"observability"
service_monitor.targets[12].url	"http://10.0.20.26:8210/health/ready"
service_monitor.targets[12].critical_for_edge	false
service_monitor.targets[12].fallback_mode	"local_logs_and_health_continue"
service_monitor.targets[13].name	"application-gateway"
service_monitor.targets[13].url	"http://10.0.20.23:8220/health/ready"
service_monitor.targets[13].critical_for_edge	false
service_monitor.targets[13].fallback_mode	"local_integrations_only"
service_monitor.targets[14].name	"media-library"
service_monitor.targets[14].url	"http://10.0.20.24:8230/health/ready"
service_monitor.targets[14].critical_for_edge	false
service_monitor.targets[14].fallback_mode	"local_media_cache_and_playout"
service_monitor.targets[15].name	"control-room"
service_monitor.targets[15].url	"http://10.0.20.25:9010/health/ready"
service_monitor.targets[15].critical_for_edge	false
service_monitor.targets[15].fallback_mode	"local_dashboard_and_audit"
service_monitor.targets[16].name	"iot-gateway"

Schlüssel	Beispielwert
service_monitor.targets[16].url	"http://10.0.20.27:8240/health/ready"
service_monitor.targets[16].critical_for_edge	false
service_monitor.targets[16].fallback_mode	"mqtt_bridge_unavailable_local_core_continues"
service_monitor.targets[17].name	"hardware-gateway"
service_monitor.targets[17].url	"http://10.0.20.28:8250/health/ready"
service_monitor.targets[17].critical_for_edge	false
service_monitor.targets[17].fallback_mode	"rack_telemetry_unavailable_local_radio_continues"
service_monitor.targets[17].depends_on	["iot-gateway"]
service_monitor.targets[18].name	"rf-monitor"
service_monitor.targets[18].url	"http://10.0.20.29:8260/health/ready"
service_monitor.targets[18].critical_for_edge	false
service_monitor.targets[18].fallback_mode	"rf_monitoring_unavailable_radio_service_continues"
service_monitor.targets[18].depends_on	["iot-gateway"]
service_monitor.targets[19].name	"alarm-workflow"
service_monitor.targets[19].url	"http://10.0.20.30:8270/health/ready"
service_monitor.targets[19].critical_for_edge	false
service_monitor.targets[19].fallback_mode	"central_alarm_escalation_unavailable_local_radio_and_sds_continue"
service_monitor.targets[19].depends_on	["iot-gateway", "sds-router"]
service_monitor.targets[20].name	"task-workflow"
service_monitor.targets[20].url	"http://10.0.20.31:8280/health/ready"
service_monitor.targets[20].critical_for_edge	false
service_monitor.targets[20].fallback_mode	"central_task_workflow_unavailable_local_radio_sds_and_cached_forms_continue"
service_monitor.targets[20].depends_on	["iot-gateway", "sds-router"]
service_monitor.targets[21].name	"asset-management"
service_monitor.targets[21].url	"http://10.0.20.32:8290/health/ready"
service_monitor.targets[21].critical_for_edge	false
service_monitor.targets[21].fallback_mode	"asset_inventory_unavailable_core_radio_services_continue"
service_monitor.targets[21].depends_on	["iot-gateway", "subscriber-core", "mobility-core",

Schlüssel	Beispielwert
	"task-workflow"]
service_monitor.targets[22].name	"sip-switch"
service_monitor.targets[22].url	"http://10.0.20.33:8300/health/ready"
service_monitor.targets[22].critical_for_edge	false
service_monitor.targets[22].fallback_mode	"central_sip_routing_unavailable_existing_local_radio_and_direct_pbx_fallback_may_continue"
service_monitor.targets[22].depends_on	["iot-gateway", "mobility-core"]

9.2 mobility-core

Quellvorlage · Ziel /etc/netcore/mobility-core.toml · Unit netcore-mobility-core.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8090"
server.history_limit	2000
server.transfer_timeout_secs	45
node_gateway.url	"ws://10.0.1.30:8080/ws/backend"
node_gateway.reconnect_secs	5
security.mode	"open_lab"
security.allow_remote_management	true
limits.max_body_bytes	1048576
limits.max_transfers	10000
limits.max_subscribers	100000

9.3 subscriber-core

Quellvorlage · Ziel /etc/netcore/subscriber-core.toml · Unit netcore-subscriber-core.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8100"
server.history_limit	2000
node_gateway.url	"ws://127.0.0.1:8080/ws/backend"
node_gateway.reconnect_secs	5
storage.database_path	"/var/lib/netcore-subscriber-core/subscribers.json"
storage.backup_path	"/var/lib/netcore-subscriber-core/subscribers.json.bak"
access_policy.mode	"allow_list"
access_policy.auto_sync	true
access_policy.disconnect_unauthorized	true
access_policy.sync_timeout_secs	30
security.mode	"open_lab"

Schlüssel	Beispielwert
security.allow_remote_management	true
limits.max_body_bytes	2097152
limits.max_subscribers	100000
limits.max_groups_per_subscriber	1024

9.4 group-core

Quellvorlage · Ziel /etc/netcore/group-core.toml · Unit netcore-group-core.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8110"
server.history_limit	2000
node_gateway.url	"ws://10.0.1.XX:8080/ws/backend"
node_gateway.reconnect_secs	5
storage.database_path	"/var/lib/netcore-group-core/groups.json"
storage.backup_path	"/var/lib/netcore-group-core/groups.json.bak"
policy.allow_unlisted_groups	false
policy.enforce_memberships	true
policy.reconcile_registered	true
policy.auto_sync	true
policy.sync_timeout_secs	30
policy.dgna_timeout_secs	30
security.mode	"open_lab"
security.allow_remote_management	true
limits.max_body_bytes	2097152
limits.max_groups	65536
limits.max_memberships	1000000

9.5 call-control

Quellvorlage · Ziel /etc/netcore/call-control.toml · Unit netcore-call-control.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8120"
server.history_limit	2000
node_gateway.url	"ws://10.0.1.XX:8080/ws/backend"
node_gateway.reconnect_secs	5
mobility_core.enabled	true
mobility_core.base_url	"http://10.0.1.XX:8090"

Schlüssel	Beispielwert
mobility_core.timeout_ms	1500
mobility_core.allow_local_fallback	false
mobility_core.accept_stale_route	false
storage.database_path	"/var/lib/netcore-call-control/calls.json"
storage.backup_path	"/var/lib/netcore-call-control/calls.json.bak"
calls.command_timeout_secs	30
calls.restore_timeout_secs	45
calls.reconcile_interval_secs	2
calls.auto_target_affiliated_nodes	true
calls.release_partial_start_on_failure	false
calls.allow_operator_force_floor	true
security.mode	"open_lab"
security.allow_remote_management	true
limits.max_body_bytes	2097152
limits.max_calls	100000
limits.max_legs_per_call	1024
limits.max_pending_commands	20000

9.6 media-switch

Quellvorlage · Ziel /etc/netcore/media-switch.toml · Unit netcore-media-switch.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8130"
server.history_limit	2000
node_gateway.url	"ws://10.0.1.20:8080/ws/backend"
node_gateway.reconnect_secs	2
call_control.url	"http://10.0.1.24:8120/api/v1/calls"
call_control.events_url	"ws://10.0.1.24:8120/ws/media"
call_control.route_ready_url	"http://10.0.1.24:8120/api/v1/media/route-ready"
call_control.reconcile_secs	15
call_control.reconnect_secs	1
call_control.request_timeout_secs	2
media.frame_duration_ms	60
media.jitter_buffer_frames	2
media.min_jitter_buffer_frames	1
media.max_jitter_buffer_frames	12
media.adaptive_jitter	true

Schlüssel	Beispielwert
media.adaptive_jitter_up_threshold_ms	18
media.adaptive_jitter_down_stable_frames	120
media.cold_start_buffer_frames	5
media.cold_start_buffer_max_age_ms	600
media.session_idle_secs	30
media.max_frames_per_tick	256
media.allow_same_leg_loopback	false
media.tap_history_frames	256
media.recorderTap_history_frames	20000
security.mode	"open_lab"
security.allow_remote_management	true
limits.max_body_bytes	1048576
limits.max_sessions	10000
limits.max_streams	50000
limits.max_pending_frames	100000

9.7 recorder

Quellvorlage · Ziel /etc/netcore/recorder.toml · Unit netcore-recorder.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8140"
server.history_limit	2000
media_switch.tap_url	"http://10.0.1.25:8130/api/v1/recorder/taps"
media_switch.sessions_url	"http://10.0.1.25:8130/api/v1/sessions"
media_switch.poll_interval_ms	100
media_switch.session_reconcile_ms	1000
media_switch.request_timeout_secs	3
media_switch.batch_limit	500
storage.root	"/var/lib/netcore-recorder/recordings"
storage.export_root	"/var/lib/netcore-recorder/exports"
storage.frame_duration_ms	60
storage.session_absent_grace_secs	3
storage.maximum_idle_secs	600
storage.default_retention_days	30
storage.retention_scan_secs	60

Schlüssel	Beispielwert
storage.fsync_every_frames	50
storage.minimum_free_space_mb	512
security.mode	"open_lab"
security.allow_remote_management	true
security.allow_delete	true
limits.max_body_bytes	1048576
limits.max_active_recordings	1000
limits.max_recordings	100000

9.8 sds-router

Quellvorlage · Ziel /etc/netcore/sds-router.toml · Unit netcore-sds-router.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8150"
server.history_limit	4000
node_gateway.url	"ws://10.0.1.20:8080/ws/backend"
node_gateway.reconnect_secs	5
storage.database_path	"/var/lib/netcore-sds-router/messages.json"
storage.backup_path	"/var/lib/netcore-sds-router/messages.json.bak"
routing.default_ttl_secs	300
routing.max_ttl_secs	86400
routing.max_attempts	5
routing.initial_retry_secs	2
routing.max_retry_secs	60
routing.dedupe_window_secs	30
routing.presence_timeout_secs	90
routing.authoritative_ingress	true
security.mode	"open_lab"
security.allow_remote_management	true
security.mask_payload_in_list	false
limits.max_body_bytes	2097152
limits.max_payload_bytes	2048
limits.max_messages	100000
limits.max_routes	4096

9.9 packet-core

Quellvorlage · Ziel /etc/netcore/packet-core.toml · Unit netcore-packet-core.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8160"
server.history_limit	5000
node_gateway.url	"ws://127.0.0.1:8080/ws/backend"
node_gateway.reconnect_secs	5
storage.database_path	"/var/lib/netcore-packet-core/state.json"
storage.backup_path	"/var/lib/netcore-packet-core/state.json.bak"
packet.mode	"shadow"
packet.ready_timer_secs	5
packet.standby_timer_secs	300
packet.response_wait_secs	10
packet.context_ready_secs	30
packet.default_mtu	1500
packet.max_n_pdu_bytes	65535
packet.max_contexts_per_subscriber	14
packet.max_total_contexts	4096
packet.strict_source_address	true
packet.preserve_context_on_node_loss	true
address_pool.network_prefix	[10, 44, 0]
address_pool.first_host	2
address_pool.last_host	254
address_pool.gateway	"10.44.0.1"
address_pool.allow_static	true
fragmentation.timeout_secs	30
fragmentation.max_datagrams	256
fragmentation.max_total_bytes	8388608
fragmentation.max_fragments_per_datagram	512
fragmentation.reject_overlaps	true
flow_control.max_queue_packets_per_context	64
flow_control.max_queue_bytes_per_context	262144
flow_control.queue_ttl_secs	30
flow_control.action_retry_secs	5
flow_control.action_max_attempts	5
security.mode	"open_lab"

Schlüssel	Beispielwert
security.allow_remote_management	true
security.expose_payloads	true
limits.max_body_bytes	1048576
limits.max_events	10000
limits.max_actions	10000
limits.max_payload_bytes	65535

9.10 ip-gateway

Quellvorlage · Ziel /etc/netcore/ip-gateway.toml · Unit netcore-ip-gateway.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8170"
packet_core.url	"http://127.0.0.1:8160"
packet_core.poll_interval_ms	250
packet_core.context_refresh_ms	1000
packet_core.request_timeout_ms	2000
packet_core.outbox_batch	250
storage.database_path	"/var/lib/netcore-ip-gateway/state.json"
storage.backup_path	"/var/lib/netcore-ip-gateway/state.json.bak"
interface.mode	"shadow"
interface.name	"ntc-tun0"
interface.address	"10.0.0.1/24"
interface.network	"10.0.0.0/24"
interface.mtu	480
interface.owner_user	"netcore"
interface.delete_on_exit	true
routing.enable_ipv4_forwarding	true
routing.reconcile_interval_secs	5
routing.install_connected_route	true
nat.enabled	true
nat.masquerade	true
nat.egress_interface	"eth0"
firewall.enabled	true
firewall.default_forward_policy	"drop"
firewall.allow_established	true
firewall.allow_general_internet	true
firewall.allow_icmp	true

Schlüssel	Beispielwert
firewall.log_drops	false
dns.enabled	true
dns.bind	"10.0.0.1:53"
dns.upstream	"1.1.1.1:53"
dns.local_domain	"netcore.test"
dns.ttl_secs	30
dns.query_timeout_ms	2000
test_server.enabled	true
test_server.bind	"0.0.0.0:8088"
test_server.udp_echo_bind	"0.0.0.0:7007"
capture.directory	"/var/lib/netcore-ip-gateway/captures"
capture.max_captures	64
capture.max_file_bytes	268435456
capture.snaplen	65535
security.mode	"open_lab"
security.allow_remote_management	true
limits.max_body_bytes	2097152
limits.max_events	100000
limits.max_flows	100000
limits.max_packet_bytes	65535

9.11 security-core

Quellvorlage · Ziel /etc/netcore/security-core.toml · Unit netcore-security-core.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8180"
server.history_limit	5000
node_gateway.url	"ws://127.0.0.1:8080/ws/backend"
node_gateway.reconnect_secs	5
node_gateway.observe_nodes	true
storage.database_path	"/var/lib/netcore-security-core/state.json"
storage.backup_path	"/var/lib/netcore-security-core/state.json.bak"
storage.lab_seed_path	"/var/lib/netcore-security-core/lab-auth.seed"
policy.operating_mode	"shadow"
policy.default_security_class	1

Schlüssel	Beispielwert
policy.minimum_security_class	1
policy.authentication_required	true
policy.allow_class1_fallback	true
policy.reject_unknown_subscribers	false
policy.disable_after_failures	false
authentication.provider	"lab_hmac_sha256"
authentication.challenge_bytes	16
authentication.response_bytes	16
authentication.challenge_ttl_secs	30
authentication.max_attempts	3
authentication.lockout_secs	300
authentication.issue_dck_on_success	true
dck.key_bytes	16
dck.ttl_secs	3600
dck.rotate_before_secs	300
dck.max_active_per_subscriber	2
security.mode	"open_lab"
security.allow_remote_management	true
security.expose_ephemeral_edge_material	true
limits.max_body_bytes	1048576
limits.max_profiles	100000
limits.max_contexts	20000
limits.max_actions	20000
limits.max_alarms	20000
limits.max_audit	100000

9.12 kmf

Quellvorlage · Ziel /etc/netcore/kmf.toml · Unit netcore-kmf.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8190"
server.history_limit	5000
storage.database_path	"/var/lib/netcore-kmf/state.json"
storage.vault_path	"/var/lib/netcore-kmf/vault.json"
storage.master_key_path	"/var/lib/netcore-kmf/master.key"
storage.backup_dir	"/var/lib/netcore-kmf/backups"

Schlüssel	Beispielwert
storage.bootstrap_dir	"/var/lib/netcore-kmf/bootstrap"
policy.operating_mode	"shadow"
policy.default_key_bytes	16
policy.default_crypto_period_secs	86400
policy.rotation_lead_secs	3600
policy.require_dual_approval	true
policy.allow_overlapping_crypto_periods	true
policy.auto_retire_predecessor	true
vault.provider	"lab_file_vault"
vault.master_key_bytes	32
vault.fsync	true
otar.action_ttl_secs	600
otar.max_attempts	5
otar.retry_backoff_secs	15
otar.max_claim_batch	100
security.mode	"open_lab"
security.allow_remote_management	true
security.expose_raw_keys	false
limits.max_body_bytes	1048576
limits.max_keys	100000
limits.max_nodes	10000
limits.max_jobs	100000
limits.max_actions	500000
limits.max_audit	100000

9.13 transit

Quellvorlage · Ziel /etc/netcore/transit.toml · Unit netcore-transit.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8200"
server.history_limit	5000
storage.database_path	"/var/lib/netcore-transit/state.json"
storage.backup_path	"/var/lib/netcore-transit/state.json.bak"
region.region_id	"region-a"
region.swmi_id	"netcore-swmi-a"
region.display_name	"NetCore Region A"

Schlüssel	Beispielwert
region.advertised_endpoint	"http://10.0.10.12:8200"
region.protocol_version	"netcore-transit-v1"
region.operating_mode	"shadow"
region.capabilities	["mobility", "individual_call", "group_call", "sds", "media", "supplementary_service"]
routing.max_hops	8
routing.dedupe_ttl_secs	900
routing.session_idle_ttl_secs	3600
routing.route_stale_secs	120
routing.prefer_direct_region_peer	true
routing.allow_transitive_routing	true
routing.allow_dynamic_peers	false
routing.fail_closed_on_loop	true
transport.connect_timeout_ms	2000
transport.io_timeout_ms	5000
transport.heartbeat_interval_secs	5
transport.peer_timeout_secs	20
transport.retry_backoff_secs	3
transport.max_attempts	5
transport.max_batch	100
security.mode	"open_lab"
security.allow_remote_management	true
security.tls	false
security.token_auth	false
limits.max_body_bytes	4194304
limits.max_peers	1000
limits.max_routes	100000
limits.max_sessions	100000
limits.max_envelopes	500000
limits.max_local_deliveries	500000
limits.max_events	100000

9.14 application-gateway

Quellvorlage · Ziel /etc/netcore/application-gateway.toml · Unit netcore-application-gateway.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8220"

Schlüssel	Beispielwert
server.public_base_url	"http://127.0.0.1:8220"
server.max_body_bytes	4194304
server.history_limit	5000
storage.state_path	"/var/lib/netcore-application-gateway/state.json"
storage.state_backup_path	"/var/lib/netcore-application-gateway/state.json.bak"
storage.secrets_path	vor Ort setzen; nicht veröffentlichen
storage.spool_dir	"/var/lib/netcore-application-gateway/spool"
storage.backup_dir	"/var/lib/netcore-application-gateway/backups"
security.mode	"open_lab"
security.management_token_auth	false
security.management_tls	false
security.allow_remote_management	true
security.connector_secrets_allowed	vor Ort setzen; nicht veröffentlichen
security.warning_banner	"OPEN LAB: no login, no management tokens and no TLS. Isolated management network only."
runtime.operating_mode	"shadow"
runtime.worker_interval_ms	1000
runtime.probe_interval_secs	30
runtime.default_ttl_secs	300
runtime.max_attempts	6
runtime.base_backoff_secs	2
runtime.max_backoff_secs	120
runtime.dedupe_window_secs	600
runtime.max_response_bytes	65536
runtime.max_artifact_bytes	33554432
runtime.max_events	20000
runtime.max_deliveries	50000
runtime.max_tts_jobs	5000
runtime.max_audit_records	50000
runtime.event_retention_secs	604800
runtime.delivery_retention_secs	1209600
runtime.audit_retention_secs	2592000
connectors[0].connector_id	"sds-router"
connectors[0].display_name	"SDS Router"
connectors[0].kind	"sds_router"

Schlüssel	Beispielwert
connectors[0].direction	"outbound"
connectors[0].endpoint	"http://127.0.0.1:8150/api/v1/messages"
connectors[0].health_endpoint	"http://127.0.0.1:8150/health/ready"
connectors[0].enabled	true
connectors[0].timeout_ms	5000
connectors[0].rate_limit_per_minute	600
connectors[0].circuit_failure_threshold	5
connectors[0].circuit_open_secs	60
connectors[0].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[0].settings.source_iss	"9999"
connectors[0].settings.sds_type	"4"
connectors[0].settings.protocol_id	"0"
connectors[0].settings.priority	"3"
connectors[1].connector_id	"piper-tts"
connectors[1].display_name	"Piper TTS"
connectors[1].kind	"piper_tts"
connectors[1].direction	"outbound"
connectors[1].endpoint	"http://127.0.0.1:5005/synthesize"
connectors[1].health_endpoint	"http://127.0.0.1:5005/voices"
connectors[1].enabled	true
connectors[1].timeout_ms	30000
connectors[1].rate_limit_per_minute	30
connectors[1].circuit_failure_threshold	3
connectors[1].circuit_open_secs	60
connectors[1].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[2].connector_id	"media-library"
connectors[2].display_name	"Media Library"
connectors[2].kind	"media_library"
connectors[2].direction	"outbound"
connectors[2].endpoint	"http://127.0.0.1:8230/api/v1/assets/import-url"
connectors[2].health_endpoint	"http://127.0.0.1:8230/health/ready"
connectors[2].enabled	true
connectors[2].timeout_ms	10000
connectors[2].rate_limit_per_minute	120

Schlüssel	Beispielwert
connectors[2].circuit_failure_threshold	5
connectors[2].circuit_open_secs	60
connectors[2].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[2].settings.method	"POST"
connectors[3].connector_id	"telegram"
connectors[3].display_name	"Telegram Bot"
connectors[3].kind	"telegram_bot"
connectors[3].direction	"bidirectional"
connectors[3].endpoint	"https://api.telegram.org/bot{bot_token}/sendMessage"
connectors[3].enabled	false
connectors[3].timeout_ms	10000
connectors[3].rate_limit_per_minute	30
connectors[3].circuit_failure_threshold	5
connectors[3].circuit_open_secs	120
connectors[3].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[3].settings.chat_id	""
connectors[3].settings.parse_mode	""
connectors[4].connector_id	"dapnet"
connectors[4].display_name	"DAPNET Relay"
connectors[4].kind	"dapnet_http"
connectors[4].direction	"bidirectional"
connectors[4].endpoint	"http://127.0.0.1:8225/api/v1/messages"
connectors[4].enabled	false
connectors[4].timeout_ms	10000
connectors[4].rate_limit_per_minute	60
connectors[4].circuit_failure_threshold	5
connectors[4].circuit_open_secs	120
connectors[4].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[4].settings.callsign	""
connectors[5].connector_id	"meshcom"
connectors[5].display_name	"MeshCom"
connectors[5].kind	"meshcom_http"
connectors[5].direction	"bidirectional"

Schlüssel	Beispielwert
connectors[5].endpoint	"http://127.0.0.1:1799/api/message"
connectors[5].enabled	false
connectors[5].timeout_ms	10000
connectors[5].rate_limit_per_minute	120
connectors[5].circuit_failure_threshold	5
connectors[5].circuit_open_secs	60
connectors[5].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[6].connector_id	"snom"
connectors[6].display_name	"Snom Notify"
connectors[6].kind	"snom_notify"
connectors[6].direction	"outbound"
connectors[6].endpoint	"http://127.0.0.1:8089/notify"
connectors[6].enabled	false
connectors[6].timeout_ms	5000
connectors[6].rate_limit_per_minute	120
connectors[6].circuit_failure_threshold	5
connectors[6].circuit_open_secs	60
connectors[6].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[7].connector_id	"geoalarm"
connectors[7].display_name	"GeoAlarm"
connectors[7].kind	"geoalarm_http"
connectors[7].direction	"bidirectional"
connectors[7].endpoint	"http://127.0.0.1:8099/api/alarms"
connectors[7].enabled	false
connectors[7].timeout_ms	10000
connectors[7].rate_limit_per_minute	60
connectors[7].circuit_failure_threshold	5
connectors[7].circuit_open_secs	120
connectors[7].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[8].connector_id	"weather"
connectors[8].display_name	"WX / METAR"
connectors[8].kind	"weather_http"
connectors[8].direction	"bidirectional"

Schlüssel	Beispielwert
connectors[8].endpoint	"https://aviationweather.gov/api/data/metar"
connectors[8].enabled	false
connectors[8].timeout_ms	10000
connectors[8].rate_limit_per_minute	60
connectors[8].circuit_failure_threshold	5
connectors[8].circuit_open_secs	120
connectors[8].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[8].settings.station	"EDDV"
connectors[9].connector_id	"tpg2200"
connectors[9].display_name	"TPG2200 Bridge"
connectors[9].kind	"tpg2200_bridge"
connectors[9].direction	"outbound"
connectors[9].endpoint	"http://127.0.0.1:8150/api/v1/messages"
connectors[9].health_endpoint	"http://127.0.0.1:8150/health/ready"
connectors[9].enabled	false
connectors[9].timeout_ms	5000
connectors[9].rate_limit_per_minute	120
connectors[9].circuit_failure_threshold	5
connectors[9].circuit_open_secs	60
connectors[9].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[9].settings.source_iss	"9999"
connectors[9].settings.protocol_id	"130"
connectors[9].settings.priority	"3"
connectors[10].connector_id	"directory"
connectors[10].display_name	"Status Directory"
connectors[10].kind	"directory_http"
connectors[10].direction	"bidirectional"
connectors[10].endpoint	"http://127.0.0.1:8060/api/v1/events"
connectors[10].enabled	false
connectors[10].timeout_ms	5000
connectors[10].rate_limit_per_minute	120
connectors[10].circuit_failure_threshold	5
connectors[10].circuit_open_secs	60

Schlüssel	Beispielwert
connectors[10].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[10].settings.method	"POST"
connectors[11].connector_id	"generic-webhook"
connectors[11].display_name	"Generic Webhook"
connectors[11].kind	"generic_webhook"
connectors[11].direction	"bidirectional"
connectors[11].endpoint	"http://127.0.0.1:9000/webhook"
connectors[11].enabled	false
connectors[11].timeout_ms	5000
connectors[11].rate_limit_per_minute	60
connectors[11].circuit_failure_threshold	5
connectors[11].circuit_open_secs	60
connectors[11].required_secrets	vor Ort setzen; nicht veröffentlichen
connectors[11].settings.method	"POST"
rules[0].rule_id	"manual-to-sds"
rules[0].name	"Manual messages to SDS Router"
rules[0].enabled	true
rules[0].priority	100
rules[0].source_connector	"manual"
rules[0].event_type	"sds.message"
rules[0].target_connector	"sds-router"
rules[0].template_id	"sds-standard"
rules[0].stop_processing	false
rules[1].rule_id	"manual-webhook"
rules[1].name	"Manual webhook dispatch"
rules[1].enabled	false
rules[1].priority	50
rules[1].source_connector	"manual"
rules[1].event_type	"webhook.message"
rules[1].target_connector	"generic-webhook"
rules[1].template_id	"generic-event-json"
rules[1].stop_processing	false
templates[0].template_id	"sds-standard"
templates[0].name	"SDS Standard"

Schlüssel	Beispielwert
templates[0].kind	"text"
templates[0].body	"{{text}}"
templates[0].content_type	"text/plain; charset=utf-8"
templates[0].enabled	true
templates[0].target_connector	"sds-router"
templates[0].description	"Plain SDS text with destination supplied by the event"
templates[1].template_id	"generic-event-json"
templates[1].name	"Generic event JSON"
templates[1].kind	"json"
templates[1].body	"{\"source\":\"{{source}}\", \"event_type\": \"{{event_type}}\", \"destination\": \"{{destination}}\", \"text\": \"{{text}}\"}"
templates[1].content_type	"application/json"
templates[1].enabled	true
templates[1].target_connector	"generic-webhook"
templates[1].description	"Small interoperable webhook envelope"
templates[2].template_id	"tts-announcement"
templates[2].name	"TTS Announcement"
templates[2].kind	"tts"
templates[2].body	"Achtung. {{text}}"
templates[2].content_type	"text/plain; charset=utf-8"
templates[2].enabled	true
templates[2].target_connector	"piper-tts"
templates[2].description	"Reusable TTS announcement prefix"

9.15 media-library

Quellvorlage · Ziel /etc/netcore/media-library.toml · Unit netcore-media-library.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8230"
server.public_base_url	"http://127.0.0.1:8230"
server.max_body_bytes	100663296
security.mode	"open_lab"
security.token_auth	false
security.tls	false
security.allow_remote_management	true
security.allow_delete	true

Schlüssel	Beispielwert
security.allow_url_import	true
security.allow_private_import_urls	true
storage.root	"/var/lib/netcore-media-library/assets"
storage.state_file	"/var/lib/netcore-media-library/state.json"
storage.temp_root	"/var/lib/netcore-media-library/tmp"
storage.backup_root	"/var/lib/netcore-media-library/backups"
storage.archive_root	"/mnt/nfs-share/Media-Library"
storage.recording_archive_root	"/mnt/nfs-share/Recordings"
storage.tts_archive_root	"/mnt/nfs-share/TTS-Dateien"
storage.max_asset_bytes	67108864
storage.max_total_bytes	21474836480
storage.fsync_imports	true
runtime.operating_mode	"shadow"
runtime.worker_interval_ms	500
runtime.probe_interval_secs	15
runtime.import_timeout_secs	120
runtime.max_assets	10000
runtime.max_jobs	2000
runtime.max_events	5000
runtime.max_audit_records	10000
runtime.max_attempts	3
runtime.frame_interval_ms	60
runtime.auto_approve_tts	false
runtime.auto_archive_recordings	true
runtime.auto_archive_tts	true
playout.mode	"basisstation"
playout.default_station	"srv-m-tbs-01"
playout.request_timeout_secs	15
playout.completion_timeout_secs	900
playout.poll_interval_ms	500
playout.stations[0].id	"srv-m-tbs-01"
playout.stations[0].name	"SRV-M-TBS-01"
playout.stations[0].base_url	"http://10.0.1.22:8080"
playout.stations[0].enabled	true
codec.frame_bytes	35

Schlüssel	Beispielwert
codec.ffmpeg_command	["/usr/bin/ffmpeg", "-hide_banner", "-loglevel", "error", "-y", "-i", "{input}", "-ac", "1", "-ar", "8000", "-c:a", "pcm_s16le", "{output}"]
codec.encoder_command	[]
codec.decoder_command	[]
tts.enabled	true
tts.endpoint	"http://127.0.0.1:5005"
tts.template_directory	"/var/lib/netcore-media-library/tts/templates"
tts.default_voice	"de-thorsten"
tts.default_speed	0.95
tts.max_text_characters	2000
tts.synthesis_timeout_secs	90
tts.max_output_file_mb	25
tts.voices[0].id	"de-thorsten"
tts.voices[0].name	"Deutsch – Thorsten (mittel)"
tts.voices[0].provider_voice	"de_DE-thorsten-medium"
tts.voices[1].id	"de-thorsten-high"
tts.voices[1].name	"Deutsch – Thorsten (hoch)"
tts.voices[1].provider_voice	"de_DE-thorsten-high"
tts.voices[2].id	"de-karlsson"
tts.voices[2].name	"Deutsch – Karlsson"
tts.voices[2].provider_voice	"de_DE-karlsson-low"
tts.voices[3].id	"de-pavoque"
tts.voices[3].name	"Deutsch – Pavoque"
tts.voices[3].provider_voice	"de_DE-pavoque-low"
tts.voices[4].id	"de-thorsten-neutral"
tts.voices[4].name	"Deutsch – Thorsten emotional (neutral)"
tts.voices[4].provider_voice	"de_DE-thorsten_emotional-medium"
tts.voices[4].speaker_id	4
dependencies.media_switch_base_url	"http://127.0.0.1:8130"
dependencies.recorder_base_url	"http://127.0.0.1:8140"
dependencies.application_gateway_base_url	"http://127.0.0.1:8220"

9.16 control-room

Quellvorlage · Ziel /etc/netcore/control-room.toml · Unit netcore-control-room.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:9010"
server.node_path	"/node"
server.ui_path	"/ui"
server.history_limit	2000
node_gateway.enabled	false
node_gateway.url	"ws://node-gateway:8080/ws/backend"
node_gateway.reconnect_secs	3
node_gateway.timeout_secs	10
node_gateway.stale_after_secs	30
persistence.enabled	true
persistence.database_path	"/var/lib/netcore-control-room/control-room.sqlite3"
persistence.persist_events	true
persistence.persist_noisy_events	false
persistence.load_recent_limit	2000
auth.enabled	false
auth.allow_health_unauthenticated	true
auth.node_token_env	""
auth.bootstrap_username_env	""
auth.bootstrap_password_env	""
auth.bootstrap_role	"admin"
federation.enabled	true
federation.poll_interval_secs	5
federation.request_timeout_ms	1200
federation.failure_threshold	3
federation.fetch_summaries	true
operations.state_path	"/var/lib/netcore-control-room/operations.json"
operations.backup_path	"/var/lib/netcore-control-room/operations.json.bak"
operations.auto_service_incidents	true
operations.incident_limit	5000
operations.shift_log_limit	10000
services[0].name	"node-gateway"
services[0].display_name	"Node Gateway"
services[0].kind	"edge"
services[0].base_url	"http://10.0.20.10:8080"
services[0].health_live	"/health/live"

Schlüssel	Beispielwert
services[0].health_ready	"/health/ready"
services[0].summary_path	"/api/v1/status"
services[0].webui_path	"/"
services[0].critical	true
services[0].enabled	true
services[1].name	"mobility-core"
services[1].display_name	"Mobility Core"
services[1].kind	"core"
services[1].base_url	"http://10.0.20.11:8090"
services[1].critical	true
services[2].name	"subscriber-core"
services[2].display_name	"Subscriber Core"
services[2].kind	"core"
services[2].base_url	"http://10.0.20.12:8100"
services[2].critical	true
services[3].name	"group-core"
services[3].display_name	"Group Core"
services[3].kind	"core"
services[3].base_url	"http://10.0.20.13:8110"
services[3].critical	true
services[4].name	"call-control"
services[4].display_name	"Call Control"
services[4].kind	"core"
services[4].base_url	"http://10.0.20.14:8120"
services[4].critical	true
services[5].name	"media-switch"
services[5].display_name	"Media Switch"
services[5].kind	"media"
services[5].base_url	"http://10.0.20.15:8130"
services[5].critical	true
services[6].name	"recorder"
services[6].display_name	"Recorder"
services[6].kind	"media"
services[6].base_url	"http://10.0.20.16:8140"
services[6].critical	false

Schlüssel	Beispielwert
services[7].name	"sds-router"
services[7].display_name	"SDS Router"
services[7].kind	"data"
services[7].base_url	"http://10.0.20.17:8150"
services[7].critical	false
services[8].name	"packet-core"
services[8].display_name	"Packet Core"
services[8].kind	"data"
services[8].base_url	"http://10.0.20.18:8160"
services[8].critical	false
services[9].name	"ip-gateway"
services[9].display_name	"IP Gateway"
services[9].kind	"data"
services[9].base_url	"http://10.0.20.19:8170"
services[9].critical	false
services[10].name	"security-core"
services[10].display_name	"Security Core"
services[10].kind	"security"
services[10].base_url	"http://10.0.20.20:8180"
services[10].critical	true
services[11].name	"kmf"
services[11].display_name	"KMF"
services[11].kind	"security"
services[11].base_url	"http://10.0.20.21:8190"
services[11].critical	false
services[12].name	"transit"
services[12].display_name	"Transit"
services[12].kind	"interworking"
services[12].base_url	"http://10.0.20.22:8200"
services[12].critical	false
services[13].name	"application-gateway"
services[13].display_name	"Application Gateway"
services[13].kind	"application"
services[13].base_url	"http://10.0.20.23:8220"
services[13].critical	false

Schlüssel	Beispielwert
services[14].name	"media-library"
services[14].display_name	"Media Library"
services[14].kind	"media"
services[14].base_url	"http://10.0.20.24:8230"
services[14].critical	false
services[15].name	"iot-gateway"
services[15].display_name	"IoT Gateway / MQTT"
services[15].kind	"integration"
services[15].base_url	"http://10.0.20.27:8240"
services[15].critical	false
services[16].name	"alarm-workflow"
services[16].display_name	"Alarm Workflow"
services[16].kind	"alarm"
services[16].base_url	"http://10.0.20.30:8270"
services[16].critical	false
services[17].name	"task-workflow"
services[17].display_name	"Task Workflow / WAP"
services[17].kind	"workflow"
services[17].base_url	"http://10.0.20.31:8280"
services[17].critical	false
services[18].name	"asset-management"
services[18].display_name	"Asset Management"
services[18].kind	"management"
services[18].base_url	"http://10.0.20.32:8290"
services[18].critical	false
services[19].name	"sip-switch"
services[19].display_name	"NetCore SIP Switch"
services[19].kind	"voice-gateway"
services[19].base_url	"http://10.0.20.33:8300"
services[19].critical	false
directory.hide_infrastructure	true
service_monitor.targets[0].name	"hardware-gateway"
service_monitor.targets[0].url	"http://10.0.20.28:8250/health/ready"
service_monitor.targets[0].critical_for_edge	false

Schlüssel	Beispielwert
service_monitor.targets[0].fallback_mode	"rack_telemetry_unavailable_local_radio_continues"
service_monitor.targets[0].depends_on	["iot-gateway"]
service_monitor.targets[1].name	"rf-monitor"
service_monitor.targets[1].url	"http://10.0.20.29:8260/health/ready"
service_monitor.targets[1].critical_for_edge	false
service_monitor.targets[1].fallback_mode	"rf_monitoring_unavailable_radio_service_continues"
service_monitor.targets[1].depends_on	["iot-gateway"]
service_monitor.targets[2].name	"alarm-workflow"
service_monitor.targets[2].url	"http://10.0.20.30:8270/health/ready"
service_monitor.targets[2].critical_for_edge	false
service_monitor.targets[2].fallback_mode	"central_alarm_escalation_unavailable_local_radio_and_sds_continue"
service_monitor.targets[2].depends_on	["iot-gateway", "sds-router"]
service_monitor.targets[3].name	"task-workflow"
service_monitor.targets[3].url	"http://10.0.20.31:8280/health/ready"
service_monitor.targets[3].critical_for_edge	false
service_monitor.targets[3].fallback_mode	"central_task_workflow_unavailable_local_radio_sds_and_cached_forms_continue"
service_monitor.targets[3].depends_on	["iot-gateway", "sds-router"]
service_monitor.targets[4].name	"asset-management"
service_monitor.targets[4].url	"http://10.0.20.32:8290/health/ready"
service_monitor.targets[4].critical_for_edge	false
service_monitor.targets[4].fallback_mode	"asset_inventory_unavailable_core_radio_services_continue"
service_monitor.targets[4].depends_on	["iot-gateway", "subscriber-core", "mobility-core", "task-workflow"]
service_monitor.targets[5].name	"sip-switch"
service_monitor.targets[5].url	"http://10.0.20.33:8300/health/ready"
service_monitor.targets[5].critical_for_edge	false
service_monitor.targets[5].fallback_mode	"central_sip_routing_unavailable_existing_local_radio_and_direct_pbx_fallback_may_continue"
service_monitor.targets[5].depends_on	["iot-gateway", "mobility-core"]

9.17 observability

Quellvorlage · Ziel `/etc/netcore/observability.toml` · Unit `netcore-observability.service`.

Schlüssel	Beispielwert
<code>server.bind</code>	<code>"0.0.0.0:8210"</code>
<code>server.history_limit</code>	<code>5000</code>
<code>server.max_body_bytes</code>	<code>4194304</code>
<code>storage.state_path</code>	<code>"/var/lib/netcore-observability/state.json"</code>
<code>storage.backup_path</code>	<code>"/var/lib/netcore-observability/state.json.bak"</code>
<code>storage.diagnostic_dir</code>	<code>"/var/lib/netcore-observability/diagnostics"</code>
<code>security.mode</code>	<code>"open_lab"</code>
<code>security.token_auth</code>	<code>false</code>
<code>security.tls</code>	<code>false</code>
<code>security.allow_remote_management</code>	<code>true</code>
<code>security.warning_banner</code>	<code>"OPEN LAB: no login, no tokens and no TLS. Isolated management network only."</code>
<code>collection.scrape_interval_secs</code>	<code>15</code>
<code>collection.request_timeout_ms</code>	<code>2000</code>
<code>collection.max_response_bytes</code>	<code>2097152</code>
<code>collection.scrape_on_start</code>	<code>true</code>
<code>collection.ingest_logs</code>	<code>true</code>
<code>collection.ingest_traces</code>	<code>true</code>
<code>retention.metric_retention_secs</code>	<code>86400</code>
<code>retention.log_retention_secs</code>	<code>604800</code>
<code>retention.trace_retention_secs</code>	<code>86400</code>
<code>retention.audit_retention_secs</code>	<code>2592000</code>
<code>retention.max_series</code>	<code>10000</code>
<code>retention.max_samples_per_series</code>	<code>5760</code>
<code>retention.max_logs</code>	<code>100000</code>
<code>retention.max_spans</code>	<code>50000</code>
<code>retention.max_alerts</code>	<code>10000</code>
<code>retention.max_audit_records</code>	<code>50000</code>
<code>stack.prometheus_url</code>	<code>"http://127.0.0.1:9090"</code>
<code>stack.grafana_url</code>	<code>"http://127.0.0.1:3000"</code>
<code>stack.loki_url</code>	<code>"http://127.0.0.1:3100"</code>
<code>stack.alertmanager_url</code>	<code>"http://127.0.0.1:9093"</code>
<code>stack.prometheus_ready_path</code>	<code>"/-/ready"</code>
<code>stack.grafana_ready_path</code>	<code>"/api/health"</code>

Schlüssel	Beispielwert
stack.loki_ready_path	"/ready"
stack.alertmanager_ready_path	"/-/ready"
targets[0].target_id	"node-gateway"
targets[0].display_name	"Node Gateway"
targets[0].service	"node-gateway"
targets[0].base_url	"http://127.0.0.1:8080"
targets[0].metrics_path	"/metrics"
targets[0].live_path	"/health/live"
targets[0].ready_path	"/health/ready"
targets[0].enabled	true
targets[0].labels.environment	"open-lab"
targets[0].labels.component	"node-gateway"
targets[1].target_id	"mobility-core"
targets[1].display_name	"Mobility Core"
targets[1].service	"mobility-core"
targets[1].base_url	"http://127.0.0.1:8090"
targets[1].metrics_path	"/metrics"
targets[1].live_path	"/health/live"
targets[1].ready_path	"/health/ready"
targets[1].enabled	true
targets[1].labels.environment	"open-lab"
targets[1].labels.component	"mobility-core"
targets[2].target_id	"subscriber-core"
targets[2].display_name	"Subscriber Core"
targets[2].service	"subscriber-core"
targets[2].base_url	"http://127.0.0.1:8100"
targets[2].metrics_path	"/metrics"
targets[2].live_path	"/health/live"
targets[2].ready_path	"/health/ready"
targets[2].enabled	true
targets[2].labels.environment	"open-lab"
targets[2].labels.component	"subscriber-core"
targets[3].target_id	"group-core"
targets[3].display_name	"Group Core"

Schlüssel	Beispielwert
targets[3].service	"group-core"
targets[3].base_url	"http://127.0.0.1:8110"
targets[3].metrics_path	"/metrics"
targets[3].live_path	"/health/live"
targets[3].ready_path	"/health/ready"
targets[3].enabled	true
targets[3].labels.environment	"open-lab"
targets[3].labels.component	"group-core"
targets[4].target_id	"call-control"
targets[4].display_name	"Call Control"
targets[4].service	"call-control"
targets[4].base_url	"http://127.0.0.1:8120"
targets[4].metrics_path	"/metrics"
targets[4].live_path	"/health/live"
targets[4].ready_path	"/health/ready"
targets[4].enabled	true
targets[4].labels.environment	"open-lab"
targets[4].labels.component	"call-control"
targets[5].target_id	"media-switch"
targets[5].display_name	"Media Switch"
targets[5].service	"media-switch"
targets[5].base_url	"http://127.0.0.1:8130"
targets[5].metrics_path	"/metrics"
targets[5].live_path	"/health/live"
targets[5].ready_path	"/health/ready"
targets[5].enabled	true
targets[5].labels.environment	"open-lab"
targets[5].labels.component	"media-switch"
targets[6].target_id	"recorder"
targets[6].display_name	"Recorder"
targets[6].service	"recorder"
targets[6].base_url	"http://127.0.0.1:8140"
targets[6].metrics_path	"/metrics"
targets[6].live_path	"/health/live"
targets[6].ready_path	"/health/ready"

Schlüssel	Beispielwert
targets[6].enabled	true
targets[6].labels.environment	"open-lab"
targets[6].labels.component	"recorder"
targets[7].target_id	"sds-router"
targets[7].display_name	"SDS Router"
targets[7].service	"sds-router"
targets[7].base_url	"http://127.0.0.1:8150"
targets[7].metrics_path	"/metrics"
targets[7].live_path	"/health/live"
targets[7].ready_path	"/health/ready"
targets[7].enabled	true
targets[7].labels.environment	"open-lab"
targets[7].labels.component	"sds-router"
targets[8].target_id	"packet-core"
targets[8].display_name	"Packet Core"
targets[8].service	"packet-core"
targets[8].base_url	"http://127.0.0.1:8160"
targets[8].metrics_path	"/metrics"
targets[8].live_path	"/health/live"
targets[8].ready_path	"/health/ready"
targets[8].enabled	true
targets[8].labels.environment	"open-lab"
targets[8].labels.component	"packet-core"
targets[9].target_id	"ip-gateway"
targets[9].display_name	"IP Gateway"
targets[9].service	"ip-gateway"
targets[9].base_url	"http://127.0.0.1:8170"
targets[9].metrics_path	"/metrics"
targets[9].live_path	"/health/live"
targets[9].ready_path	"/health/ready"
targets[9].enabled	true
targets[9].labels.environment	"open-lab"
targets[9].labels.component	"ip-gateway"
targets[10].target_id	"security-core"
targets[10].display_name	"Security Core"

Schlüssel	Beispielwert
targets[10].service	"security-core"
targets[10].base_url	"http://127.0.0.1:8180"
targets[10].metrics_path	"/metrics"
targets[10].live_path	"/health/live"
targets[10].ready_path	"/health/ready"
targets[10].enabled	true
targets[10].labels.environment	"open-lab"
targets[10].labels.component	"security-core"
targets[11].target_id	"kmf"
targets[11].display_name	"KMF"
targets[11].service	"kmf"
targets[11].base_url	"http://127.0.0.1:8190"
targets[11].metrics_path	"/metrics"
targets[11].live_path	"/health/live"
targets[11].ready_path	"/health/ready"
targets[11].enabled	true
targets[11].labels.environment	"open-lab"
targets[11].labels.component	"kmf"
targets[12].target_id	"transit"
targets[12].display_name	"Transit"
targets[12].service	"transit"
targets[12].base_url	"http://127.0.0.1:8200"
targets[12].metrics_path	"/metrics"
targets[12].live_path	"/health/live"
targets[12].ready_path	"/health/ready"
targets[12].enabled	true
targets[12].labels.environment	"open-lab"
targets[12].labels.component	"transit"
targets[13].target_id	"application-gateway"
targets[13].display_name	"Application Gateway"
targets[13].service	"application-gateway"
targets[13].base_url	"http://127.0.0.1:8220"
targets[13].metrics_path	"/metrics"
targets[13].live_path	"/health/live"
targets[13].ready_path	"/health/ready"

Schlüssel	Beispielwert
targets[13].enabled	true
targets[13].labels.environment	"open-lab"
targets[13].labels.component	"application-gateway"
targets[14].target_id	"media-library"
targets[14].display_name	"Media Library"
targets[14].service	"media-library"
targets[14].base_url	"http://127.0.0.1:8230"
targets[14].metrics_path	"/metrics"
targets[14].live_path	"/health/live"
targets[14].ready_path	"/health/ready"
targets[14].enabled	true
targets[14].labels.environment	"open-lab"
targets[14].labels.component	"media-library"
targets[15].target_id	"control-room"
targets[15].display_name	"Control Room"
targets[15].service	"control-room"
targets[15].base_url	"http://127.0.0.1:9010"
targets[15].metrics_path	"/metrics"
targets[15].live_path	"/health/live"
targets[15].ready_path	"/health/ready"
targets[15].enabled	true
targets[15].labels.environment	"open-lab"
targets[15].labels.component	"control-room"
targets[16].target_id	"iot-gateway"
targets[16].display_name	"IoT Gateway / MQTT"
targets[16].service	"iot-gateway"
targets[16].base_url	"http://127.0.0.1:8240"
targets[16].metrics_path	"/metrics"
targets[16].live_path	"/health/live"
targets[16].ready_path	"/health/ready"
targets[16].enabled	true
targets[16].labels.environment	"open-lab"
targets[16].labels.component	"iot-gateway"
targets[17].target_id	"alarm-workflow"
targets[17].display_name	"Alarm Workflow"

Schlüssel	Beispielwert
targets[17].service	"alarm-workflow"
targets[17].base_url	"http://10.0.20.30:8270"
targets[17].metrics_path	"/metrics"
targets[17].live_path	"/health/live"
targets[17].ready_path	"/health/ready"
targets[17].enabled	true
targets[17].labels.environment	"open-lab"
targets[17].labels.component	"alarm-workflow"
targets[18].target_id	"task-workflow"
targets[18].display_name	"Task Workflow / WAP"
targets[18].service	"task-workflow"
targets[18].base_url	"http://10.0.20.31:8280"
targets[18].metrics_path	"/metrics"
targets[18].live_path	"/health/live"
targets[18].ready_path	"/health/ready"
targets[18].enabled	true
targets[18].labels.environment	"open-lab"
targets[18].labels.component	"task-workflow"
targets[19].target_id	"asset-management"
targets[19].display_name	"Asset Management"
targets[19].service	"asset-management"
targets[19].base_url	"http://10.0.20.32:8290"
targets[19].metrics_path	"/metrics"
targets[19].live_path	"/health/live"
targets[19].ready_path	"/health/ready"
targets[19].enabled	true
targets[19].labels.environment	"open-lab"
targets[19].labels.component	"asset-management"
targets[20].target_id	"sip-switch"
targets[20].display_name	"NetCore SIP Switch"
targets[20].service	"sip-switch"
targets[20].base_url	"http://10.0.20.33:8300"
targets[20].metrics_path	"/metrics"
targets[20].live_path	"/health/live"
targets[20].ready_path	"/health/ready"

Schlüssel	Beispielwert
targets[20].enabled	true
targets[20].labels.environment	"open-lab"
targets[20].labels.component	"sip-switch"
alert_rules[0].rule_id	"target-down"
alert_rules[0].name	"Target down"
alert_rules[0].description	"A monitored target does not answer its liveness endpoint"
alert_rules[0].metric	"netcore_observability_target_up"
alert_rules[0].comparator	"<"
alert_rules[0].threshold	1.0
alert_rules[0].for_secs	30
alert_rules[0].severity	"critical"
alert_rules[0].enabled	true
alert_rules[0].labels.source	"netcore-observability"
alert_rules[0].annotations.runbook	"Check service process, network path and /health/live"
alert_rules[1].rule_id	"target-not-ready"
alert_rules[1].name	"Target not ready"
alert_rules[1].description	"A service is alive but reports not ready"
alert_rules[1].metric	"netcore_observability_target_ready"
alert_rules[1].comparator	"<"
alert_rules[1].threshold	1.0
alert_rules[1].for_secs	60
alert_rules[1].severity	"warning"
alert_rules[1].enabled	true
alert_rules[1].labels.source	"netcore-observability"
alert_rules[1].annotations.runbook	"Check dependencies and /health/ready"
alert_rules[2].rule_id	"repeated-scrape-errors"
alert_rules[2].name	"Repeated scrape errors"
alert_rules[2].description	"The NMS cannot collect metrics from a target repeatedly"
alert_rules[2].metric	"netcore_observability_target_consecutive_failures"
alert_rules[2].comparator	">="
alert_rules[2].threshold	3.0
alert_rules[2].for_secs	0
alert_rules[2].severity	"warning"

Schlüssel	Beispielwert
alert_rules[2].enabled	true
alert_rules[2].labels.source	"netcore-observability"
alert_rules[2].annotations.runbook	"Check the target /metrics endpoint and NMS routing"

9.18 iot-gateway

Quellvorlage · Ziel /etc/netcore/iot-gateway.toml · Unit netcore-iot-gateway.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8240"
server.history_limit	1000
server.max_body_bytes	1048576
security.mode	"open_lab"
security.allow_remote_management	true
mqtt.host	"127.0.0.1"
mqtt.port	1883
mqtt.client_id	"netcore-iot-gateway"
mqtt.topic_prefix	"netcore/v1"
mqtt.keep_alive_secs	30
mqtt.clean_session	false
mqtt.reconnect_secs	3
mqtt.publish_timeout_secs	8
mqtt.qos	1
mqtt.event_retain	false
mqtt.state_retain	true
mqtt.observe_commands	true
mqtt.execute_commands	false
home_assistant.enabled	true
home_assistant.discovery_enabled	true
home_assistant.discovery_prefix	"homeassistant"
home_assistant.status_topic	"homeassistant/status"
home_assistant.node_id	"netcore_tetra"
home_assistant.discovery_qos	1
home_assistant.discovery_retain	true
home_assistant.expose_gateway	true
home_assistant.expose_sources	true
home_assistant.expose_virtual_devices	true
home_assistant.accept_state_ingress	true

Schlüssel	Beispielwert
home_assistant.state_ingress_topic	""
home_assistant.allow_command_egress	false
home_assistant.command_egress_topic	""
homematic.enabled	false
homematic.mode	"home_assistant_mqtt"
homematic.ccu_host	"127.0.0.1"
homematic.ccu_port	2010
homematic.poll_interval_ms	2000
homematic.request_timeout_ms	2500
homematic.allow_writes	false
commands.enabled	true
commands.mode	"open_lab_sandbox"
commands.default_deny	true
commands.allow_retained	false
commands.default_ttl_secs	30
commands.max_ttl_secs	300
commands.max_future_skew_secs	30
commands.publish_lifecycle_acks	true
commands.ack_qos	1
commands.ack_retain	false
storage.state_dir	"/var/lib/netcore-iot-gateway"
storage.outbox_dir	"outbox"
storage.dedup_file	"dedup.json"
storage.command_inbox_file	"command-inbox.ndjson"
storage.command_ledger_file	"command-ledger.json"
storage.command_audit_file	"command-audit.ndjson"
storage.virtual_state_file	"virtual-device-state.json"
storage.external_state_file	"external-entity-state.json"
storage.homematic_state_file	"homematic-datapoint-state.json"
storage.dedup_limit	50000
storage.outbox_limit	20000
storage.command_ledger_limit	50000
polling.interval_ms	2000
polling.batch_limit	500

Schlüssel	Beispielwert
polling.request_timeout_ms	2500
command_policies[0].id	"allow-openlab-virtual-relays"
command_policies[0].enabled	true
command_policies[0].effect	"allow"
command_policies[0].command_types	["virtual.relay.set"]
command_policies[0].target_types	["virtual_relay"]
command_policies[0].target_prefixes	["lab-"]
command_policies[0].max_ttl_secs	120
command_policies[0].allow_dry_run	true
command_policies[1].id	"allow-openlab-virtual-lights"
command_policies[1].enabled	true
command_policies[1].effect	"allow"
command_policies[1].command_types	["virtual.light.set"]
command_policies[1].target_types	["virtual_light"]
command_policies[1].target_prefixes	["lab-"]
command_policies[1].max_ttl_secs	120
command_policies[1].allow_dry_run	true
command_policies[2].id	"allow-openlab-virtual-buttons"
command_policies[2].enabled	true
command_policies[2].effect	"allow"
command_policies[2].command_types	["virtual.button.press"]
command_policies[2].target_types	["virtual_button"]
command_policies[2].target_prefixes	["lab-"]
command_policies[2].max_ttl_secs	60
command_policies[2].allow_dry_run	true
command_policies[3].id	"allow-openlab-homeassistant-lab-bridge"
command_policies[3].enabled	false
command_policies[3].effect	"allow"
command_policies[3].command_types	["homeassistant.entity.command"]
command_policies[3].target_types	["home_assistant_entity"]
command_policies[3].target_prefixes	["input_boolean.netcore_lab_"]
command_policies[3].max_ttl_secs	60
command_policies[3].allow_dry_run	true
command_policies[4].id	"allow-openlab-homematic-lab-writes"
command_policies[4].enabled	false

Schlüssel	Beispielwert
command_policies[4].effect	"allow"
command_policies[4].command_types	["homematic.datapoint.set"]
command_policies[4].target_types	["homematic_datapoint"]
command_policies[4].target_prefixes	["lab-"]
command_policies[4].max_ttl_secs	60
command_policies[4].allow_dry_run	true
sources[0].id	"node-gateway"
sources[0].url	"http://node-gateway:8080/api/v1/events/netcore"
sources[0].enabled	true
sources[1].id	"mobility-core"
sources[1].url	"http://mobility-core:8090/api/v1/events/netcore"
sources[1].enabled	true
sources[2].id	"call-control"
sources[2].url	"http://call-control:8120/api/v1/events/netcore"
sources[2].enabled	true
sources[3].id	"sds-router"
sources[3].url	"http://sds-router:8150/api/v1/events/netcore"
sources[3].enabled	true

9.19 hardware-gateway

Quellvorlage · Ziel /etc/netcore/hardware-gateway.toml · Unit netcore-hardware-gateway.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8250"
security.mode	"open_lab"
mqtt.host	"127.0.0.1"
mqtt.port	1883
mqtt.topic_prefix	"netcore/v1"
mqtt.client_id	"netcore-hardware-gateway"
storage.state_file	"/var/lib/netcore-hardware-gateway/state.json"
storage.event_log	"/var/lib/netcore-hardware-gateway/events.ndjson"
monitoring.heartbeat_timeout_secs	30
monitoring.stale_after_secs	20
monitoring.outputs_enabled	false
thresholds[0].metric	"temperature_c"
thresholds[0].warning_above	40.0
thresholds[0].critical_above	55.0

Schlüssel	Beispielwert
thresholds[1].metric	"humidity_percent"
thresholds[1].warning_above	75.0
thresholds[1].critical_above	90.0
thresholds[2].metric	"supply_voltage_v"
thresholds[2].warning_below	11.5
thresholds[2].critical_below	10.8

9.20 rf-monitor

Quellvorlage · Ziel /etc/netcore/rf-monitor.toml · Unit netcore-rf-monitor.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8260"
security.mode	"open_lab"
mqtt.enabled	true
mqtt.host	"127.0.0.1"
mqtt.port	1883
mqtt.topic_prefix	"netcore/v1"
mqtt.client_id	"netcore-rf-monitor"
storage.state_file	"/var/lib/netcore-rf-monitor/state.json"
storage.event_log	"/var/lib/netcore-rf-monitor/events.ndjson"
monitoring.heartbeat_timeout_secs	20
monitoring.event_memory_limit	1000
monitoring.max_spectrum_bins	512
monitoring.max_payload_bytes	524288
thresholds.vswr_warning	1.8
thresholds.vswr_critical	2.5
thresholds.reflected_ratio_warning_percent	10.0
thresholds.reflected_ratio_critical_percent	20.0
thresholds.pa_temp_warning_c	70.0
thresholds.pa_temp_critical_c	85.0
thresholds.sdr_temp_warning_c	65.0
thresholds.sdr_temp_critical_c	80.0
thresholds.cabinet_temp_warning_c	45.0
thresholds.cabinet_temp_critical_c	60.0

Schlüssel	Beispielwert
thresholds.evm_warning_pct	6.0
thresholds.evm_critical_pct	10.0
thresholds.papr_warning_db	7.0
thresholds.papr_critical_db	9.0
thresholds.forward_power_warning_below_w	0.0
thresholds.forward_power_critical_below_w_w	0.0
thresholds.pa_voltage_warning_below_v	0.0
thresholds.pa_voltage_critical_below_v	0.0
thresholds.fan_warning_below_rpm	0
thresholds.fan_critical_below_rpm	0

9.21 alarm-workflow

Quellvorlage · Ziel `/etc/netcore/alarm-workflow.toml` · Unit `netcore-alarm-workflow.service`.

Schlüssel	Beispielwert
server.bind	"0.0.0.0:8270"
security.mode	"open_lab"
storage.state_file	"/var/lib/netcore-alarm-workflow/state.json"
storage.event_log	"/var/lib/netcore-alarm-workflow/events.ndjson"
storage.audit_log	"/var/lib/netcore-alarm-workflow/audit.ndjson"
mqtt.enabled	true
mqtt.host	"10.0.20.27"
mqtt.port	1883
mqtt.qos	1
mqtt.topic_prefix	"netcore/v1"
mqtt.client_id	"netcore-alarm-workflow"
mqtt.reconnect_secs	3
mqtt.subscribe_topics	["netcore/v1/events/#"]
sds_router.base_url	"http://10.0.20.17:8150"
sds_router.timeout_secs	3
sds_router.poll_interval_secs	2
sds_router.process_existing_events	false
sds_router.source_issi	9999
sds_router.protocol_id	130
sds_router.default_ttl_secs	300

Schlüssel	Beispielwert
workflow.scheduler_interval_secs	2
workflow.event_history_limit	2000
workflow.seen_event_limit	10000
workflow.stop_escalation_on_ack	true
workflow.stop_escalation_on_assignment	true
workflow.auto_close_on_clear	false
limits.max_body_bytes	2097152
recipients[0].id	"technik-gruppe"
recipients[0].name	"Technikgruppe"
recipients[0].enabled	true
recipients[0].kind	"group_sds"
recipients[0].destination	15201
recipients[0].source_issi	9999
recipients[0].protocol_id	130
recipients[0].priority	7
recipients[0].ttl_secs	300
recipients[0].max_text_chars	180
recipients[0].message_template	"ALARM {severity} {token} {title}. ACK {token}"
recipients[1].id	"leitstelle-test"
recipients[1].name	"Leitstelle Testteilnehmer"
recipients[1].enabled	false
recipients[1].kind	"individual_sds"
recipients[1].destination	4010001
recipients[1].source_issi	9999
recipients[1].protocol_id	130
recipients[1].priority	10
recipients[1].ttl_secs	300
recipients[1].max_text_chars	180
recipients[1].message_template	"ESK {severity} {token} {title}. TAKE {token}"
escalation_profiles[0].id	"technical-default"
escalation_profiles[0].name	"Technische Standardescalation"
escalation_profiles[0].steps[0].after_secs	0
escalation_profiles[0].steps[0].recipients	["technik-gruppe"]
escalation_profiles[0].steps[1].after_secs	60

Schlüssel	Beispielwert
secs	
escalation_profiles[0].steps[1].recipients	["technik-gruppe"]
escalation_profiles[0].steps[2].after_secs	180
escalation_profiles[0].steps[2].recipients	["technik-gruppe", "leitstelle-test"]
escalation_profiles[1].id	"urgent"
escalation_profiles[1].name	"Dringende Eskalation"
escalation_profiles[1].steps[0].after_secs	0
escalation_profiles[1].steps[0].recipients	["technik-gruppe", "leitstelle-test"]
escalation_profiles[1].steps[1].after_secs	30
escalation_profiles[1].steps[1].recipients	["technik-gruppe", "leitstelle-test"]
escalation_profiles[1].steps[2].after_secs	90
escalation_profiles[1].steps[2].recipients	["technik-gruppe", "leitstelle-test"]
rules[0].id	"rf-alarm-raised"
rules[0].enabled	true
rules[0].action	"raise"
rules[0].event_type	"rf.alarm_raised"
rules[0].alarm_type	"rf_fault"
rules[0].title	"RF {subject_id}: {payload_alarm_key}"
rules[0].description	"HF-Alarm {payload_alarm_key}; Wert {payload_value}"
rules[0].severity	"inherit"
rules[0].priority	8
rules[0].requires_ack	true
rules[0].recipients	["technik-gruppe"]
rules[0].escalation_profile	"technical-default"
rules[0].dedup_fields	["subject.id", "payload.alarm_key"]
rules[1].id	"rf-alarm-cleared"
rules[1].enabled	true
rules[1].action	"clear"
rules[1].event_type	"rf.alarm_cleared"
rules[1].alarm_type	"rf_fault"

Schlüssel	Beispielwert
rules[1].title	"RF {subject_id}: {payload_alarm_key}"
rules[1].description	"HF-Alarm aufgehoben"
rules[1].dedup_fields	["subject.id", "payload.alarm_key"]
rules[1].auto_close	false
rules[2].id	"rf-station-offline"
rules[2].enabled	true
rules[2].action	"raise"
rules[2].event_type	"rf.station_offline"
rules[2].alarm_type	"rf_station_offline"
rules[2].title	"RF-Monitoring {subject_id} offline"
rules[2].description	"Keine RF-Telemetrie mehr empfangen"
rules[2].severity	"warning"
rules[2].priority	7
rules[2].requires_ack	true
rules[2].recipients	["technik-gruppe"]
rules[2].escalation_profile	"technical-default"
rules[2].dedup_fields	["subject.id"]
rules[3].id	"rf-station-online"
rules[3].enabled	true
rules[3].action	"clear"
rules[3].event_type	"rf.station_online"
rules[3].alarm_type	"rf_station_offline"
rules[3].title	"RF-Monitoring {subject_id} wieder online"
rules[3].dedup_fields	["subject.id"]
rules[4].id	"hardware-threshold-raised"
rules[4].enabled	true
rules[4].action	"raise"
rules[4].event_type	"hardware.threshold_exceeded"
rules[4].alarm_type	"hardware_threshold"
rules[4].title	"{subject_id}: {payload_metric} außerhalb Grenzwert"
rules[4].description	"Messwert {payload_metric} = {payload_value} ({payload_reason})"
rules[4].severity	"inherit"
rules[4].priority	7
rules[4].requires_ack	true

Schlüssel	Beispielwert
rules[4].recipients	["technik-gruppe"]
rules[4].escalation_profile	"technical-default"
rules[4].dedup_fields	["subject.id", "payload.metric"]
rules[5].id	"hardware-threshold-cleared"
rules[5].enabled	true
rules[5].action	"clear"
rules[5].event_type	"hardware.threshold_cleared"
rules[5].alarm_type	"hardware_threshold"
rules[5].title	"{subject_id}: {payload_metric} wieder normal"
rules[5].dedup_fields	["subject.id", "payload.metric"]
rules[6].id	"hardware-input-raised"
rules[6].enabled	true
rules[6].action	"raise"
rules[6].event_type	"hardware.input_activated"
rules[6].alarm_type	"hardware_input"
rules[6].title	"{subject_id}: Eingang {payload_input} aktiv"
rules[6].description	"Hardware-Eingang {payload_input} wurde aktiv"
rules[6].severity	"inherit"
rules[6].priority	10
rules[6].requires_ack	true
rules[6].recipients	["technik-gruppe", "leitstelle-test"]
rules[6].escalation_profile	"urgent"
rules[6].dedup_fields	["subject.id", "payload.input"]
rules[7].id	"hardware-input-cleared"
rules[7].enabled	true
rules[7].action	"clear"
rules[7].event_type	"hardware.input_cleared"
rules[7].alarm_type	"hardware_input"
rules[7].title	"{subject_id}: Eingang {payload_input} wieder inaktiv"
rules[7].dedup_fields	["subject.id", "payload.input"]
rules[8].id	"hardware-device-offline"
rules[8].enabled	true
rules[8].action	"raise"
rules[8].event_type	"hardware.device_offline"

Schlüssel	Beispielwert
rules[8].alarm_type	"hardware_offline"
rules[8].title	"Hardware-Node {subject_id} offline"
rules[8].description	"Heartbeat des Hardware-Nodes fehlt"
rules[8].severity	"warning"
rules[8].priority	6
rules[8].requires_ack	true
rules[8].recipients	["technik-gruppe"]
rules[8].escalation_profile	"technical-default"
rules[8].dedup_fields	["subject.id"]
rules[9].id	"hardware-device-online"
rules[9].enabled	true
rules[9].action	"clear"
rules[9].event_type	"hardware.device_online"
rules[9].alarm_type	"hardware_offline"
rules[9].title	"Hardware-Node {subject_id} wieder online"
rules[9].dedup_fields	["subject.id"]
rules[10].id	"node-disconnected"
rules[10].enabled	true
rules[10].action	"raise"
rules[10].event_type	"node.disconnected"
rules[10].alarm_type	"node_offline"
rules[10].title	"TBS/Node {subject_id} getrennt"
rules[10].description	"Node Gateway meldet Verbindungsverlust"
rules[10].severity	"critical"
rules[10].priority	10
rules[10].requires_ack	true
rules[10].recipients	["technik-gruppe", "leitstelle-test"]
rules[10].escalation_profile	"urgent"
rules[10].dedup_fields	["subject.id"]
rules[11].id	"node-connected"
rules[11].enabled	true
rules[11].action	"clear"
rules[11].event_type	"node.connected"
rules[11].alarm_type	"node_offline"
rules[11].title	"TBS/Node {subject_id} wieder verbunden"

Schlüssel	Beispielwert
rules[11].dedup_fields	["subject.id"]
status_actions[0].enabled	true
status_actions[0].status_code	5201
status_actions[0].action	"ack"
status_actions[0].states	["open"]
status_actions[0].requires_ack_only	true
status_actions[1].enabled	true
status_actions[1].status_code	5202
status_actions[1].action	"take"
status_actions[1].states	["open", "acknowledged"]
status_actions[2].enabled	true
status_actions[2].status_code	5203
status_actions[2].action	"start"
status_actions[2].states	["open", "acknowledged", "assigned"]
status_actions[3].enabled	true
status_actions[3].status_code	5204
status_actions[3].action	"resolve"
status_actions[3].states	["open", "acknowledged", "assigned", "in_progress"]
status_actions[4].enabled	true
status_actions[4].status_code	5205
status_actions[4].action	"close"
status_actions[4].states	["resolved"]

9.22 task-workflow

Quellvorlage · Ziel /etc/netcore/task-workflow.toml · Unit netcore-task-workflow.service.

Schlüssel	Beispielwert
service.name	"netcore-task-workflow"
service.phase	9
service.mode	"open_lab"
server.bind	"0.0.0.0:8280"
security.mode	"open_lab"
storage.state_file	"/var/lib/netcore-task-workflow/state.json"
storage.event_log	"/var/lib/netcore-task-workflow/events.ndjson"
storage.audit_log	"/var/lib/netcore-task-workflow/audit.ndjson"
mqtt.enabled	true
mqtt.host	"127.0.0.1"

Schlüssel	Beispielwert
mqtt.port	1883
mqtt.topic_prefix	"netcore/v1"
mqtt.client_id	"netcore-task-workflow"
sds_router.enabled	true
sds_router.base_url	"http://127.0.0.1:8150"
sds_router.source_issi	9999
sds_router.protocol_id	130
sds_router.ttl_secs	600
sds_router.max_text_length	160
sds_router.default_destination	15201
sds_router.default_is_group	true
workflow.event_history_limit	2000
workflow.seen_event_limit	10000
workflow.expire_check_interval_secs	5
workflow.notify_on_state_change	true
wap.enabled	true
wap.page_size	6
wap.xhtml_entry	"/x"
wap.wml_entry	"/w"
templates[0].id	"technical_fault"
templates[0].name	"Technische Stoerung"
templates[0].description	"Technische Stoerung aufnehmen und bearbeiten"
templates[0].default_priority	7
templates[0].default_severity	"warning"
templates[0].requires_ack	true
templates[0].fields[0].id	"asset"
templates[0].fields[0].label	"Anlage/Geraet"
templates[0].fields[0].type	"text"
templates[0].fields[0].required	true
templates[0].fields[1].id	"fault"
templates[0].fields[1].label	"Fehlerbild"
templates[0].fields[1].type	"text"
templates[0].fields[1].required	true
templates[0].fields[2].id	"location"

Schlüssel	Beispielwert
templates[0].fields[2].label	"Ort"
templates[0].fields[2].type	"text"
templates[0].fields[2].required	false
templates[1].id	"vehicle_check"
templates[1].name	"Fahrzeugcheck"
templates[1].description	"Kurzer Fahrzeug- und Ausrüstungscheck"
templates[1].default_priority	3
templates[1].default_severity	"notice"
templates[1].requires_ack	true
templates[1].fields[0].id	"vehicle"
templates[1].fields[0].label	"Fahrzeug"
templates[1].fields[0].type	"text"
templates[1].fields[0].required	true
templates[1].fields[1].id	"mileage"
templates[1].fields[1].label	"Kilometer"
templates[1].fields[1].type	"number"
templates[1].fields[1].required	false
templates[1].fields[2].id	"result"
templates[1].fields[2].label	"Ergebnis"
templates[1].fields[2].type	"text"
templates[1].fields[2].required	true
templates[2].id	"material_withdrawal"
templates[2].name	"Materialentnahme"
templates[2].description	"Materialentnahme dokumentieren"
templates[2].default_priority	2
templates[2].default_severity	"info"
templates[2].requires_ack	false
templates[2].fields[0].id	"item"
templates[2].fields[0].label	"Material"
templates[2].fields[0].type	"text"
templates[2].fields[0].required	true
templates[2].fields[1].id	"quantity"
templates[2].fields[1].label	"Menge"
templates[2].fields[1].type	"number"
templates[2].fields[1].required	true

Schlüssel	Beispielwert
templates[2].fields[2].id	"purpose"
templates[2].fields[2].label	"Verwendung"
templates[2].fields[2].type	"text"
templates[2].fields[2].required	false
templates[3].id	"check_in"
templates[3].name	"Check-in"
templates[3].description	"Person, Fahrzeug oder Team einchecken"
templates[3].default_priority	2
templates[3].default_severity	"info"
templates[3].requires_ack	false
templates[3].fields[0].id	"resource"
templates[3].fields[0].label	"Ressource"
templates[3].fields[0].type	"text"
templates[3].fields[0].required	true
templates[3].fields[1].id	"location"
templates[3].fields[1].label	"Ort"
templates[3].fields[1].type	"text"
templates[3].fields[1].required	true
templates[4].id	"check_out"
templates[4].name	"Check-out"
templates[4].description	"Person, Fahrzeug oder Team auschecken"
templates[4].default_priority	2
templates[4].default_severity	"info"
templates[4].requires_ack	false
templates[4].fields[0].id	"resource"
templates[4].fields[0].label	"Ressource"
templates[4].fields[0].type	"text"
templates[4].fields[0].required	true
templates[4].fields[1].id	"result"
templates[4].fields[1].label	"Rueckmeldung"
templates[4].fields[1].type	"text"
templates[4].fields[1].required	false
templates[5].id	"maintenance_ack"
templates[5].name	"Wartungsquittierung"
templates[5].description	"Wartung oder Pruefung bestaetigen"

Schlüssel	Beispielwert
templates[5].default_priority	4
templates[5].default_severity	"notice"
templates[5].requires_ack	true
templates[5].fields[0].id	"asset"
templates[5].fields[0].label	"Anlage/Geraet"
templates[5].fields[0].type	"text"
templates[5].fields[0].required	true
templates[5].fields[1].id	"work"
templates[5].fields[1].label	"Arbeit"
templates[5].fields[1].type	"text"
templates[5].fields[1].required	true
templates[5].fields[2].id	"result"
templates[5].fields[2].label	"Ergebnis"
templates[5].fields[2].type	"text"
templates[5].fields[2].required	true
status_actions[0].status_code	5301
status_actions[0].action	"accept"
status_actions[0].label	"Auftrag annehmen"
status_actions[1].status_code	5302
status_actions[1].action	"start"
status_actions[1].label	"Bearbeitung beginnen"
status_actions[2].status_code	5303
status_actions[2].action	"block"
status_actions[2].label	"Auftrag blockiert"
status_actions[3].status_code	5304
status_actions[3].action	"complete"
status_actions[3].label	"Auftrag erledigt"
status_actions[4].status_code	5305
status_actions[4].action	"cancel"
status_actions[4].label	"Auftrag abbrechen"

9.23 asset-management

Quellvorlage · Ziel /etc/netcore/asset-management.toml · Unit netcore-asset-management.service.

Schlüssel	Beispielwert
service.name	"netcore-asset-management"
service.phase	10

Schlüssel	Beispielwert
service.mode	"open_lab"
server.bind	"0.0.0.0:8290"
security.mode	"open_lab"
storage.state_file	"/var/lib/netcore-asset-management/state.json"
storage.event_log	"/var/lib/netcore-asset-management/events.ndjson"
storage.audit_log	"/var/lib/netcore-asset-management/audit.ndjson"
mqtt.enabled	true
mqtt.host	"127.0.0.1"
mqtt.port	1883
mqtt.topic_prefix	"netcore/v1"
mqtt.client_id	"netcore-asset-management"
management.event_history_limit	3000
management.upstream_sync_interval_secs	60
upstreams.subscriber_core.enabled	true
upstreams.subscriber_core.base_url	"http://127.0.0.1:8100"
upstreams.mobility_core.enabled	true
upstreams.mobility_core.base_url	"http://127.0.0.1:8090"
upstreams.task_workflow.enabled	true
upstreams.task_workflow.base_url	"http://127.0.0.1:8280"
upstreams.task_workflow.default_gssi	15201

9.24 sip-switch

Quellvorlage · Ziel /etc/netcore/sip-switch.toml · Unit netcore-sip-switch.service.

Schlüssel	Beispielwert
service.name	"netcore-sip-switch"
service.phase	11
service.mode	"open_lab"
server.bind	"0.0.0.0:8300"
security.mode	"open_lab"
storage.state_file	"/var/lib/netcore-sip-switch/state.json"
storage.event_log	"/var/lib/netcore-sip-switch/events.ndjson"
storage.audit_log	"/var/lib/netcore-sip-switch/audit.ndjson"
mqtt.enabled	true
mqtt.host	"127.0.0.1"

Schlüssel	Beispielwert
mqtt.port	1883
mqtt.topic_prefix	"netcore/v1"
mqtt.client_id	"netcore-sip-switch"
mobility_core.enabled	true
mobility_core.base_url	"http://127.0.0.1:8090"
mobility_core.timeout_secs	2
asterisk.enabled	true
asterisk.binary	"/usr/sbin/asterisk"
asterisk.config_dir	"/etc/asterisk"
asterisk.agi_script	"/var/lib/asterisk/agi-bin/netcore-sip-route.py"
asterisk.sip_bind	"0.0.0.0:5060"
asterisk.rtp_start	10000
asterisk.rtp_end	20000
management.route_workers	8
management.side_effect_queue_size	512
management.call_history_limit	2000
management.event_history_limit	3000
management.probe_interval_secs	10
pbx.mode	"registration"
pbx.endpoint_id	"netcore-pbx"
pbx.host	"127.0.0.1"
pbx.port	5060
pbx.transport	"udp"
pbx.username	""
pbx.auth_username	""
pbx.password	""
pbx.from_user	"netcore-tetra"
pbx.from_domain	""
pbx.contact_user	"netcore-tetra"
pbx.registration_id	"netcore-pbx-registration"
pbx.registration_expiration_secs	30
pbx.allow	"ulaw"
pbx.match	[]
routing.tetra_number_prefix	""
routing.strip_tetra_prefix	false

Schlüssel	Beispielwert
routing.pb_x_outbound_prefix	""
routing.strip_pb_x_outbound_prefix	false
routing.accept_stale_routes	false
routing.require_tbs_contact	true
routing.dial_timeout_secs	60
tbs[0].node_id	"SRV-M-TBS-01"
tbs[0].endpoint_id	"tbs-srv-m-tbs-01"
tbs[0].username	"tbs-srv-m-tbs-01"
tbs[0].password	vor Ort setzen; nicht veröffentlichen
tbs[0].enabled	false
tbs[0].max_contacts	1
tbs[0].aliases	["TBS-01"]
number_mappings[0].number	"4010001"
number_mappings[0].target_type	"issi"
number_mappings[0].target	4010001
number_mappings[0].enabled	false

9.25 alert-service

Quellvorlage · Ziel /etc/netcore/alert-service.toml · Unit netcore-alert-service.service.

Schlüssel	Beispielwert
server.bind	"0.0.0.0"
server.port	8310
server.admin_token	""
server.allow_unauthenticated	false
storage.database	"/var/lib/netcore-alert-service/alerts.sqlite3"
netcore.control_room_url	"http://control-room:9010"
netcore.sds_router_url	"http://sds-router:8150"
netcore.control_room_username	""
netcore.control_room_password	""
netcore.source_issi	9999
netcore.poll_seconds	5
netcore.gps_max_age_seconds	3600
netcore.node_max_age_seconds	120
netcore.http_timeout_seconds	10
nina.enabled	true
nina.base_url	"https://warnung.bund.de/api31"

Schlüssel	Beispielwert
nina.sources	["mowas", "katwarn", "biwapp", "dwd", "lhp"]
nina.poll_seconds	60
nina.max_stale_seconds	300
delivery.enabled	false
delivery.ttl_seconds	300
delivery.max_text_length	120

10. Betriebs- und Bedienabläufe

10.1 Start- und Stopp-Reihenfolge

Start: Node Gateway → Mobility/Subscriber → Group → Call/Packet/Security → Media/SDS/IP/KMF → Transit/Application/Recorder/Media Library → Control Room → Observability. Das Inventory berechnet diese Reihenfolge automatisch.

Beim geplanten Komplettstopp umgekehrt vorgehen. Die Basisstation kann lokal weiterlaufen; für Wartung am Core ist ein bewusstes degraded/isolated-Fenster daher zulässig.

10.2 Teilnehmer anlegen

1. Subscriber-Core-WebUI öffnen.
2. ISSI, Name, Organisation, Home-MCC/MNC, Status enabled und registration_allowed setzen.
3. Sprach-, SDS- und Packet-Data-Berechtigungen passend aktivieren.
4. Speichern und TBS-Sync prüfen.
5. Funkgerät registrieren; Live-Lage und Node Gateway kontrollieren.
6. Bei allow_list ist ein fehlendes Profil eine Ablehnung - das ist beabsichtigt.

10.3 Gruppe und Mitgliedschaft

1. Group Core: GSSI, Name, Priorität/Class of Usage und Dienstfreigaben anlegen.
2. Teilnehmer als feste oder dynamische Mitglieder hinzufügen.
3. Optional Auto-Affiliation definieren.
4. DGNA erst nach erfolgreichem Policy-Sync auslösen.
5. Affiliation in Group Core und TBS prüfen.

10.4 Gruppenruf

1. Teilnehmer registriert und affiliert?
2. Call Control: logischen Gruppenruf starten oder Ruf von Funkgerät initiieren.
3. Call Legs und Floor Holder prüfen.
4. Media Switch: Sessions, Streams und Jitter-Puffer kontrollieren.
5. Recorder: aktive Aufnahme und später Integrität prüfen.
6. Ruf sauber beenden; keine Testframe-Injection in belegtem Netz verwenden.

10.5 SDS senden

1. SDS Router: Zielart Individual/Group, Ziel-SSI, Typ, Protocol-ID, Priorität und TTL setzen.
2. Nachricht absenden und Zustelllegs beobachten.
3. Gewöhnliche SDS können Store-and-forward verwenden. Warnungen aus alert-service nutzen dagegen die gesonderte At-most-once-Politik mit Ablaufzeit; Anhang B beachten.
4. Application Gateway kann Vorlagen, Webhooks oder TTS-Workflows auslösen; im Shadow-Modus entstehen keine externen Nebenwirkungen.

10.6 Packet Data

1. Packet Core und IP Gateway zunächst Shadow.
2. Adresspool/Netz/MTU konsistent prüfen.
3. IP Gateway Kernel-Plan prüfen; TUN-Passthrough und nftables bereitstellen.
4. Packet Core auf authoritative, danach IP Gateway auf authoritative umstellen.
5. PDP-Kontext aktivieren, Lease prüfen, WAP-Testseite <http://10.0.0.1:8088/> beziehungsweise das konfigurierte Gateway testen.
6. PCAP nur zeitlich und größenmäßig begrenzt aktivieren.

10.7 Aufnahme, TTS und Aussendung

1. Recorder nimmt automatisch aus dem Media-Switch-Tap auf.
2. Application Gateway erzeugt TTS-WAV über Piper.
3. Media Library importiert WAV/MP3/TACELP; Vorschau und Metadaten prüfen.
4. Asset freigeben. WAV/MP3 brauchen für Funkbereitschaft einen echten TETRA-Encoder; .tace1p muss positives Vielfaches von 35 Byte sein.
5. Für Payout muss bereits eine Media-Switch-Session existieren.
6. Job starten, Fortschritt beobachten, bei Fehler bewusst ab Frame 0 wiederholen.

10.8 Security und KMF

1. Security Core zunächst Shadow; Profile/Policy beobachten.
2. Lab-HMAC ist nur Integrationsprovider, keine produktive TETRA-Authentisierung.
3. KMF-Keys und Crypto Periods in Shadow erzeugen/prüfen.
4. Vier-Augen-Freigaben im Open Lab sind nur deklarativ.
5. Authoritative/OTAR erst aktivieren, wenn der Air-Interface-Adapter und sichere Schlüsselwege nachweislich funktionieren.
6. Bei Core-Ausfall keine neuen Schlüssel erfinden; bereits installierte Schlüssel bleiben lokal nutzbar.

10.9 Control Room und Observability

Der Control Room ist die erste Lageansicht. Für fachliche Änderungen in die jeweilige Service-WebUI wechseln. Incidents quittieren, Notizen ergänzen, Ursache beheben und erst dann lösen. In Observability nach Trace-/Correlation-ID suchen, Scrape Targets testen und bei Wartungen Silences setzen.

11. Offline-Fallback der Basisstation

11.1 Auslöser

Die TBS pingt keinen öffentlichen Internetdienst. Entscheidend sind ihre WebSocket-Verbindung zum Node Gateway und die vollständige, revisionierte Core-Service-Matrix. Damit führen Internet-, VPN-, Routing- oder Core-Ausfälle zum selben klaren Ergebnis.

Zustand	Bedeutung
online	Gateway und alle benötigten Dienste gesund.
degraded	Einzelne Dienste ausgefallen; nur deren lokaler Fallback aktiv.
isolated	Gateway nicht erreichbar oder Matrix-Lease abgelaufen; TBS lokal autoritativ.
recovering	Dienste wieder gesund; Hysterese und Replay laufen.

11.2 Lokale Funktionen

Ausgefallener Dienst	Lokales Verhalten
Node Gateway	lokale Edge-Autonomie; Reconnect läuft weiter
Subscriber Core	letzte Policy, dann statische Konfiguration
Group Core	letzte Gruppenpolicy und lokale Affiliationen
Mobility Core	lokale Registrierung und Location Area
Call Control	Rufe innerhalb der lokalen Zelle
Media Switch	lokale Air-Interface-Medien; keine zentralen Frames
Recorder	lokaler Recorder läuft weiter
SDS Router	lokale Zustellung; nicht-lokale Nachrichten persistent gequeued
Packet Core	lokale SNDCCP/PDP-Kontexte
IP Gateway	lokales TUN/Routing, sofern auf TBS konfiguriert
Security Core	letzte Policy; kein stilles Downgrade
KMF	installierte Keys; kein erfundenes OTAR
Transit	kein Inter-Region-Routing
Control Room	lokales TBS-Dashboard und Audit
Observability	lokale Logs/Health
Application Gateway	nur lokal erreichbare Integrationen
Media Library	lokal gecachte/freigegebene Medien

11.3 Diagnose

```
# Basisstation
curl -fsS http://127.0.0.1:8080/api/edge-fallback | jq .
```

```
# Node Gateway
curl -fsS http://10.0.20.10:8080/api/v1/core-services | jq .
```

Auf der TBS sind insbesondere gateway_connected, service_matrix_fresh, mode, reason, services, policy_cache und event_spool zu prüfen.

11.4 Recovery

1. Ursache im Netz/Core beheben.
2. Node Gateway muss wieder vollständige gesunde Matrix liefern.
3. recover_after_secs abwarten; nicht durch wiederholte Neustarts flappen lassen.
4. Replay-Spool beobachten. Sprachframes werden absichtlich nicht nachträglich ausgesendet.
5. Subscriber-/Group-Policy-Sync und SDS-Remotelegs prüfen.
6. Erst nach online wieder zentrale Operatoraktionen auslösen.

12. Backup, Update und Wiederherstellung

12.1 Basisstation sichern

```
sudo systemctl stop tetra.service
sudo tar -C / -czf /var/backups/netcore-tbs-$(date +%Y%m%d-%H%M%S).tar.gz \
  etc/netcore var/lib/netcore var/lib/flowstation var/cache/netcore
sudo systemctl start tetra.service
```

Große Aufzeichnungen/Archive separat behandeln.

12.2 LXC-State sichern

Mindestens Konfiguration, /var/lib/<dienst>, systemd-Unit und ggf. /opt/<dienst> sichern. KMF-Master-Key separat und offline sichern. Für Recorder/Media Library Archivdaten, für IP Gateway PCAPs, für Observability Diagnose/Retention berücksichtigen.

12.3 Update der Basisstation

```
sudo systemctl stop tetra.service
cd /opt/netcore-tetra
git status
git pull --ff-only          # bei Git-Installation
git rev-parse HEAD
cargo build --locked --release -p bluestation-bs
sudo install -m 0755 target/release/bluestation-bs /usr/local/bin/bluestation-bs
sudo systemctl start tetra.service
sudo journalctl -u tetra.service -n 200 --no-pager
```

12.4 Update eines LXC-Dienstes

```
sudo cp <config> <config>.pre-update
sudo system-backend/<dienst>/install/update.sh
sudo systemctl status <unit> --no-pager
curl -i http://127.0.0.1:<port>/health/ready
```

Oder das vorher geprüfte Deployment-Bundle erneut über den Deployer anwenden.

12.5 Rollback

1. Dienst stoppen.
2. Vorherige Binär-/Quellversion wiederherstellen.
3. Passende alte Konfiguration zurückkopieren.
4. State nur dann zurückrollen, wenn Schema/Upgrade dies erfordert; vorher aktuellen State sichern.
5. Dienst manuell starten, Logs prüfen, dann systemd aktivieren.

13. Systemtest und Abnahme

13.1 Statische Vorprüfung

```
python3 tools/check_full_system_integration.py
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --profile full
--validate-only
```

13.2 Smoke-Test

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --profile smoke
```

Prüfziel: Liveness, Status, dienstspezifische API-Verträge und WebUIs sowie Mock-TBS und Control-Room-Polling. Die alte Full-System-Prüfliste erwartet noch 24 Dienste und Metrics/OpenAPI auch für alert-service; diese drei aktuellen Auditbefunde sind in Anhang B aufgeführt.

13.3 Vollständiger Funktionstest

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml \
test --profile full --allow-mutations --timeout 35
```

Prüft Teilnehmer/Gruppe, Gruppenruf, Media/Recorder, SDS Store-and-forward, Packet Data, Federation und Observability. Nur in einem leeren Labornetz ausführen.

13.4 Fault- und Fallback-Test

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml \
test --profile fault --allow-mutations --allow-restarts --timeout 45
```

Dieser Test stoppt Dienste absichtlich. Abnahme: failed=0, alle Opferrdienste wieder active/ready, TBS sieht unavailable und anschließende Recovery, keine Testfixtures bleiben liegen.

13.5 On-Air-Abnahme

Mock-Tests belegen keine HF-, MAC-, LLC-, MLE-, MM- oder CMCE-Konformität. Mit mindestens zwei Geräteherstellern dokumentieren: MCC/MNC/LAC/Colour Code/Carrier, Registrierung, Gruppenruf, SDS, Packet Data, Audio, Release/Restore, Zeitstempel, Softwarestände und Evidenzdateien.

14. Fehlersuche

14.1 Standardreihenfolge

1. Fehlerzeitpunkt, ISSI/GSSI, TBS und Correlation-ID notieren.
2. `systemctl status` und **erste** konkrete Journal-Fehlermeldung lesen.
3. Liveness, Readiness und Abhängigkeiten getrennt prüfen.
4. Nur eine Variable ändern.
5. Nach Fix den vollständigen Ablauf inklusive Release wiederholen.

14.2 Schnelldiagnose

```
systemctl status <unit> --no-pager
journalctl -u <unit> -b -n 300 --no-pager
ss -ltnup
curl -v http://127.0.0.1:<port>/health/live
curl -v http://127.0.0.1:<port>/health/ready
curl -fsS http://127.0.0.1:<port>/api/v1/status | jq .
```

Symptom	Wahrscheinliche Ursache	Prüfung
TBS startet mit Fallback	TOML-Fehler/Primärdatei unlesbar	Journal nach config/fallback/parse durchsuchen; diff zur Fallback-Datei
Node im Gateway nicht sichtbar	falscher host/port/path, Firewall, Node-ID-Duplikat	TBS [control_room], WebSocket /ws/node, Gateway-Events
Core-Dienst live aber nicht ready	Abhängigkeit nicht erreichbar oder Shadow-/Kernel-Voraussetzung	Status JSON und gerenderte URL prüfen
Gruppenruf ohne Sprache	keine Media-Session, falsche Legs, Jitter/Frameformat	Call Control Legs, Media Switch RX/TX/Drops
Recorder leer	Tap-URL falsch oder Cursor-Lücke	Media-Switch Recorder-Tap und Recorder-Events
SDS bleibt queued	Ziel nicht präsent, keine Route, zentrale SDS-Funktion nicht aktiv	Presence, TBS-Sync, TTL/Retry
Packet Data ohne Internet	IP Gateway shadow, TUN fehlt, NAT/Firewall/DNS falsch	kernel/plan, ip addr/route, nft ruleset
Control Room zeigt alte Config	falscher Config-Pfad	/etc/netcore-control-room/control-room.toml und ExecStart prüfen
Media-Vorschau fehlt	ffmpeg/Decoder fehlt oder Dateiformat ungültig	Media-Library-Logs und Codec-Konfig
Fallback bleibt recovering	Matrix nicht vollständig gesund oder Replay offen	TBS /api/edge-fallback, Gateway /api/v1/core-services
NFS blockiert/readonly	Mountoptionen, UID/GID, NAS nicht erreichbar	mount, findmnt, touch als Dienstuser

15. Inbetriebnahme-Checkliste

15.1 Infrastruktur

- ☐ Management-VLAN isoliert; keine Portweiterleitung
- ☐ 25 statische LXC-Adressen eingetragen
- ☐ NTP/DNS/Hosts funktionieren
- ☐ SSH-Key-Deployment getestet
- ☐ IP-Gateway-TUN vorhanden
- ☐ NFS/Archiv mit nofail und korrekten Rechten
- ☐ Backupspeicher getrennt vorhanden

15.2 Basisstation

- ☐ SDR erkannt und Passband geprüft
- ☐ Frequenzen/MCC/MNC/LAC/Colour Code genehmigt und korrekt
- ☐ config.toml und config.toml.fallback getrennt geprüft
- ☐ Dashboard-Passwort geändert
- ☐ Node-ID eindeutig
- ☐ Node Gateway erreichbar
- ☐ Edge-Fallback online/degraded/isolated/recovering getestet
- ☐ lokaler Ruf/SDS bei abgetrenntem Core erfolgreich

15.3 Core

- ☐ Inventory mit korrigiertem Control-Room-Pfad
- ☐ validate/plan/render ohne Fehler
- ☐ alle Dienste live
- ☐ Readiness-Ausfälle erklärbar
- ☐ Shadow-Dienste noch nicht unbeabsichtigt authoritative
- ☐ Subscriber/Group Policy synchron
- ☐ Recorder/Media Storage geprüft
- ☐ KMF-Master-Key separat gesichert
- ☐ Control Room federiert alle Dienste
- ☐ Observability Targets grün

15.4 Abnahme

- ☐ Smoke failed=0
- ☐ Full failed=0
- ☐ Fault failed=0
- ☐ keine Testfixtures übrig
- ☐ On-Air-Test dokumentiert
- ☐ Rollback einmal praktisch getestet
- ☐ Betriebs- und Notfallkontakte dokumentiert

Anhang A: Port-, Pfad- und Befehlsreferenz

A.1 Dienste

Dienst	Port	Unit	Konfiguration	Persistenz
node-gateway	8080	netcore-node-gateway.service	/etc/netcore/node-gateway.toml	In-Memory; keine Fachdatenbank
mobility-core	8090	netcore-mobility-core.service	/etc/netcore/mobility-core.toml	aktuell primär Laufzeitlage
subscriber-core	8100	netcore-subscriber-core.service	/etc/netcore/subscriber-core.toml	/var/lib/netcore-subscriber-core/subscribers.json
group-core	8110	netcore-group-core.service	/etc/netcore/group-core.toml	/var/lib/netcore-group-core/groups.json
call-control	8120	netcore-call-control.service	/etc/netcore/call-control.toml	/var/lib/netcore-call-control/calls.json
media-switch	8130	netcore-media-switch.service	/etc/netcore/media-switch.toml	zeitkritische Laufzeitdaten im Speicher
recorder	8140	netcore-recorder.service	/etc/netcore/recorder.toml	/var/lib/netcore-recorder/recordings
sds-router	8150	netcore-sds-router.service	/etc/netcore/sds-router.toml	/var/lib/netcore-sds-router/messages.json
packet-core	8160	netcore-packet-core.service	/etc/netcore/packet-core.toml	/var/lib/netcore-packet-core/state.json
ip-gateway	8170	netcore-ip-gateway.service	/etc/netcore/ip-gateway.toml	/var/lib/netcore-ip-gateway/state.json und captures/
security-core	8180	netcore-security-core.service	/etc/netcore/security-core.toml	/var/lib/netcore-security-core/state.json; Seed separat
kmf	8190	netcore-kmf.service	/etc/netcore/kmf.toml	/var/lib/netcore-kmf/state.json + vault.json + master.key
transit	8200	netcore-transit.service	/etc/netcore/transit.toml	/var/lib/netcore-transit/state.json
application-gateway	8220	netcore-application-gateway.service	/etc/netcore/application-gateway.toml	state.json, secrets.json, spool, backups
media-library	8230	netcore-media-library.service	/etc/netcore/media-library.toml	/var/lib/netcore-media-library/assets + state.json
control-room	9010	netcore-control-room.service	/etc/netcore/control-room.toml	control-room.sqlite3 + operations.json
observability	8210	netcore-observability.service	/etc/netcore/observability.toml	/var/lib/netcore-observability/state.json + diagnostics/
iot-gateway	8240	netcore-iot-gateway.service	/etc/netcore/iot-gateway.toml	/var/lib/netcore-iot-gateway, outbox, dedup.json, command-inbox.ndjson, command-

Dienst	Port	Unit	Konfiguration	Persistenz
				ledger.json, command-audit.ndjson, virtual-device-state.json, external-entity-state.json, homematic-datapoint-state.json
hardware-gateway	8250	netcore-hardware-gateway.service	/etc/netcore/hardware-gateway.toml	/var/lib/netcore-hardware-gateway/state.json, /var/lib/netcore-hardware-gateway/events.ndjson
rf-monitor	8260	netcore-rf-monitor.service	/etc/netcore/rf-monitor.toml	/var/lib/netcore-rf-monitor/state.json, /var/lib/netcore-rf-monitor/events.ndjson
alarm-workflow	8270	netcore-alarm-workflow.service	/etc/netcore/alarm-workflow.toml	/var/lib/netcore-alarm-workflow/state.json, /var/lib/netcore-alarm-workflow/events.ndjson, /var/lib/netcore-alarm-workflow/audit.ndjson
task-workflow	8280	netcore-task-workflow.service	/etc/netcore/task-workflow.toml	/var/lib/netcore-task-workflow/state.json, /var/lib/netcore-task-workflow/events.ndjson, /var/lib/netcore-task-workflow/audit.ndjson
asset-management	8290	netcore-asset-management.service	/etc/netcore/asset-management.toml	/var/lib/netcore-asset-management/state.json, /var/lib/netcore-asset-management/events.ndjson, /var/lib/netcore-asset-management/audit.ndjson
sip-switch	8300	netcore-sip-switch.service	/etc/netcore/sip-switch.toml	/var/lib/netcore-sip-switch/state.json, /var/lib/netcore-sip-switch/events.ndjson, /var/lib/netcore-sip-switch/audit.ndjson
alert-service	8310	netcore-alert-service.service	/etc/netcore/alert-service.toml	/var/lib/netcore-alert-service/alerts.sqlite3

A.2 Zusätzliche Ports

Komponente	Port	Zweck
Piper TTS	5005/tcp	lokale Sprachsynthese
IP Gateway DNS	53/udp	DNS Forwarder
IP Gateway Testserver	8088/tcp	HTTP/WAP/WML
IP Gateway UDP Echo	7007/udp	UDP-Test
Grafana	3000/tcp	Dashboards

Komponente	Port	Zweck
Prometheus	9090/tcp	Metriken
Alertmanager	9093/tcp	Alarmrouting
Loki	3100/tcp	Logs

A.3 Standardbefehle

Logs

```
journalctl -u <unit> -f
journalctl -u <unit> -b -n 300 --no-pager
```

Health

```
curl -fsS http://<ip>:<port>/health/live
curl -i http://<ip>:<port>/health/ready
curl -fsS http://<ip>:<port>/metrics | head
```

Basisstation-Fallback

```
curl -fsS http://<tbs>:8080/api/edge-fallback | jq .
curl -fsS http://<node-gateway>:8080/api/v1/core-services | jq .
```

Deployment/E2E

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml status
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --profile smoke
```

A.4 Mitgelieferte Begleitdateien

- `deploy/open-lab/inventory.example.toml` - aktuelle 25-Dienst-Vorlage; Control-Room-Zielpfad gegen die aktive Unit prüfen.
- `Docs/basisstation.config.sanitized.example.toml` - vollständige, bereinigte Basisstationsvorlage ohne ursprüngliche Zugangsdaten.
- `NetCore-Tetra-Komplettinguide-2026-09-27.md` - editierbare Markdown-Quelle dieser Ausgabe.

Anhang B Warnzentrale und v1.9.0 Betrieb

Diese Ausgabe dokumentiert Release **v1.9.0** auf main bei 086a81fa8820ef579c475a65a38e3d23644c52f0. Stichtag ist der **27. September 2026 in Europe/Berlin**; die Veröffentlichung erfolgte am 26. September um 23:02 UTC. Die neue Warnzentrale erweitert das Open-Lab-Inventory auf **25 deploybare Dienste**. system-backend/services.toml zählt **26 Einträge einschließlich der Bibliothek shared**. Provisioning Core ist ein zusätzlicher Workspace-Dienst und steht in keiner dieser beiden Listen.

Gegenüber den bisherigen Vorlagen sind NINA/KATWARN, eigene Warnungen, GPS-gestützte Einzel-SDS, die Control-Room-Telemetrie vom Node Gateway sowie die SDS-, Call-Control-, Frame-18- und SIP-Reparaturen berücksichtigt. Bestehende Konfigurationen und Persistenzdaten bleiben beim Update erhalten. Ein erfolgreicher Build ersetzt keine Ende-zu-Ende- oder Funkabnahme.

Die Referenzprüfung ergab OK: 25 services, contract=netcore.v1, mode=open_lab. Der Full-System-Audit ergab **FAIL mit drei Befunden**: Die Prüfliste erwartet noch 24 Dienste; alert-service bietet die erwarteten /metrics- und /openapi.json-Marker nicht; die Root-TBS-Konfiguration verwendet für das Gateway 10.0.1.179, das Inventory 10.0.20.10. Der separat bestätigte GitHub-Workflow **Warning service** für diesen Commit ist erfolgreich. Diese Aussagen betreffen Quellstand, lokale statische Prüfung und CI; laufende TBS, LXC und Funkgeräte wurden nicht geprüft.

Warnzentrale und Versandweg

alert-service läuft im Beispiel auf 10.0.20.34:8310, nutzt Python 3.11 oder neuer ohne zusätzliche pip-Pakete und speichert Warnungen sowie Versandreservierungen in SQLite. Er liest Warnungen der Quellen mowas, katwarn, biwapp, dwd und lhp über die öffentliche BBK/NINA-Schnittstelle. Das ist keine gesonderte KATWARN-Partneranbindung.

Der Versandweg lautet: Warnquelle oder eigene Meldung → Warnzentrale → Teilnehmer-/GPS-Lage im Control Room → SDS Router → Node Gateway → aktuelle TBS → Funkgerät. Für ein Ziel müssen Registrierung, ausreichend frische GPS-Daten und eine verbundene, aktuelle TBS belegt sein. Ohne diese Voraussetzungen wird das Gerät nicht einfach mit einer veralteten Position beschickt.

Installation und Aktualisierung

Für einen neuen Aufbau in dieser Reihenfolge vorgehen:

1. SDS Router auf v1.9.0 aktualisieren und /health/ready sowie /api/v1/status prüfen.
2. Control Room aktualisieren und dessen lesenden Node-Gateway-Kanal auf denselben Gateway wie den Router konfigurieren.
3. Warn-LXC mit dem aktuellen Installer einrichten; Token, Ziel-URLs, DNS und Datenpfad prüfen.
4. Betroffene TBS aktualisieren, damit DeliverSds und SendStatus im CMCE-Pfad verarbeitet werden.
5. Nach Anmeldung eine neue GPS-Meldung erzeugen und in Control Room sowie Warnzentrale verfolgen. Der Gateway liefert beim ersten Verbinden keinen vollständigen alten GPS-Snapshot.
6. Zunächst mit delivery.enabled = false prüfen. Versand erst nach einem kontrollierten Test gezielt einschalten.

Bei einer bereits funktionierenden Warnanlage benötigt eine reine Erweiterung der Warn-WebUI nur ein Update des Warn-LXC. Die getrennten SDS-Router-, Call-Control- und TBS-Reparaturen werden nach ihrem jeweiligen Fehlerbild ausgerollt. Der Node Gateway benötigt für den Warnpfad keinen zusätzlichen Funktionspatch.

Die verbindlichen, vollständigen Updatebefehle stehen in Docs/KATWARN_NINA_INSTALL_UPDATE.md, Docs/SDS_ROUTER_STATUS_ABSTURZ_REPARATUR.md, Docs/SDS_CALL_CONTROL_FALLBACK_REPARATUR.md und Docs/SEPURA_REREG_FRAME18_BNCH.md des festgehaltenen Releases. Vor jedem Installer aktive Unit, tatsächlichen Config-Pfad und Sicherung prüfen.

Konfiguration und Authentisierung

Unit: netcore-alert-service.service. Benutzer: netcore-alert. Konfiguration: /etc/netcore/alert-service.toml. Token-Umgebung: /etc/netcore/alert-service.env mit NETCORE_ALERT_TOKEN; bei authentisiertem Control Room gegebenenfalls NETCORE_CONTROL_ROOM_PASSWORD. Der Installer erzeugt das Token lokal. Es gehört nicht in Git oder diese Dokumentation.

Die Warnzentrale ist eine Ausnahme vom offenen Management vieler anderer Labordienste: allow_unauthenticated = false; die Fach-API verlangt ein Bearer-Token. /health/live und /health/ready bleiben für Health-Proben erreichbar. Direkte HTTP-Listener weiterhin im geschützten Managementnetz halten.

Einstellung	Vorlage	Bedeutung
netcore.control_room_url	http://control-room:9010	Quelle für Teilnehmer und GPS
netcore.sds_router_url	http://sds-router:8150	Zentraler Versandweg
netcore.poll_seconds	5	Intervall der Lageabfrage
netcore.gps_max_age_seconds	3600	Höchstalter der GPS-Position
netcore.node_max_age_seconds	120	Höchstalter der TBS-Lage
nina.poll_seconds	60	Abrufintervall der Warnquellen
nina.max_stale_seconds	300	Toleranz für veraltete Warndaten
delivery.enabled	false	Versand bleibt zunächst deaktiviert
delivery.ttl_seconds	300	Begrenzte Lebensdauer einer Sendung
delivery.max_text_length	120	Begrenzung des Warntexts
storage.database	/var/lib/netcore-alert-service/alerts.sqlite3	Warnungen und Versandreservierungen

Im Control Room ergänzt [node_gateway] den lesenden Telemetripfad. Die Vorlage verwendet enabled = false, url = "ws://node-gateway:8080/ws/backend", reconnect_secs = 3, timeout_secs = 10 und stale_after_secs = 30. Erst mit korrekter Gateway-Adresse aktivieren. Diesen Backend-Pfad nicht mit dem TBS-Pfad /ws/node verwechseln.

Bedienung und Zustellsemantik

Die WebUI trennt alle, aktive und beendete Meldungen und bietet Suche. Eigene Warnungen besitzen Titel, Text, Schweregrad, Mittelpunkt, Radius von 50 Metern bis 200 Kilometern und eine begrenzte Laufzeit von höchstens 366 Tagen. Löschen beendet künftige Aussendungen; die Historie bleibt erhalten. Bereits gesendete SDS lassen sich nicht zurückrufen. Beim Beenden wird keine zusätzliche Entwarnungs-SDS erzeugt.

Die Ansicht device_overview enthält auch ausgeschlossene Geräte mit Begründung. Die kompakte Liste devices enthält die nutzbaren Geräte. area_status unterscheidet affected, outside, no_alerts und unknown. Die UI-Kategorie „Prüfung nötig“ kann zusätzlich Versandprobleme enthalten und ist daher nicht immer identisch mit dem reinen Blockiert-Zähler.

Warnung und Ziel-SSSI werden persistent reserviert. Der Router erhält idempotency_key, at_most_once = true und eine absolute Ablaufzeit. Wiederholungen und Neustarts sollen keine doppelte Warnung auslösen. Bei unklarer Übergabe wird zugunsten der Duplikatvermeidung nicht blind erneut gesendet. **accepted bedeutet Annahme durch die TBS zum Senden und ist keine Empfangs- oder Lesebestätigung des Funkgeräts.** Der gewöhnliche Store-and-forward-Pfad anderer SDS wird durch diese spezielle Warnpolitik nicht pauschal ersetzt.

API	Zweck
GET /api/v1/status	Gesamtzustand und Geräteübersicht
GET /api/v1/alerts	Warnungen lesen
POST /api/v1/alerts	Eigene Warnung anlegen
DELETE /api/v1/alerts/{id}	Künftigen Versand beenden
GET /api/v1/devices	Geräteinformationen

API	Zweck
GET /api/v1/deliveries	Versandhistorie

Alle aufgeführten Fachrouten verlangen das konfigurierte Token. Diese Ausgabe veröffentlicht keine echten Tokens oder Standortdaten.

Sicherung und Fehlersuche

Die Warn-SQLite-Datenbank und den konfigurierten SDS-Router-Persistenzstand gemeinsam mit Konfiguration und Releasekennung sichern. Eine laufende SQLite-Datenbank mit konsistentem Backupverfahren sichern oder den Dienst für eine Kopie geordnet anhalten. Bei Rückkehr zu einem alten Build die Daten nicht löschen: Sonst können bereits behandelte Warnungen erneut ausgelöst werden. Ein Rollback des Programms ist nicht automatisch ein kompatibler Rollback seines Datenschemas.

Fehlt ein Gerät, Registrierung, GPS-Alter, TBS-Verbindung und `device_overview` prüfen. Bei HTTP 401 Token und Header prüfen. Bei fehlender Lage den neuen Control-Room-Gateway-Kanal und eine frische GPS-Meldung kontrollieren. Bei `live=200`, aber abbreißendem `ready` zuerst nach `index out of bounds` und `PoisonError` im SDS-Router-Journal suchen. Die v1.9.0-Korrektur prüft kurze Status-Payloads vor dem Zugriff; ein Neustart allein behebt den alten Fehler nicht.

Weitere Reparaturen des Releases

SDS Router und Call Control schreiben unveränderte Gateway-Telemetrie nicht mehr unnötig unter einer globalen Zustandssperre auf Disk. Das adressiert langsame Status-/Readiness-Antworten und dadurch ausgelöste Fallbacks. Am konkreten Standort müssen Latenz und Recovery trotzdem gemessen werden.

Die TBS korrigiert den obligatorischen BNCH in Rahmen 18 auf Kontroll- und belegten Verkehrskanälen. Der Patch belegt weder die Ursache aller Sepura-Neuanmeldungen noch vollständige Rahmen-18-Konformität. Ein reproduzierbarer Vorher-/Nachher-Funktest bleibt erforderlich.

Die vom SIP Switch erzeugten Asterisk-Includes benötigen Leserechte für die Gruppe `asterisk`; v1.9.0 setzt dafür `0640` auch unter `umask 077`. Diese gezielte Ausnahme nicht durch eine pauschale `0600`-Regel wieder unlesbar machen. `Cargo.lock` wurde für gesperrte Builds repariert; die Workspace-Paketversion allein ist weiterhin keine Releasekennung.

Vergleich mit den drei Upstream-Projekten

Der erste Lauf setzt eine feste Vergleichsbasis. In den ausgewerteten Standardbranches gab es in den sieben Tagen vor diesem Stichtag keine neuen Commits. Die folgenden Unterschiede sind vorhandener Integrationsrückstand und keine neu veröffentlichten Wochenänderungen. Die Bewertung ist ein gezielter Quellvergleich, keine pauschale Aussage über alle Upstream-Commits.

Quelle und Stand	Geprüfter Unterschied	Bewertung für NetCore Tetra
FlowStation 0f4faa98, 24.07.2026	Abbrechen eines vorgemerkten Circuit-Close beim erneuten Öffnen	Übernahmekandidat: NetCore öffnet den Circuit ohne entsprechende Löschung in <code>rx_control_circuit_open</code> ; an acht logische Slots und Dual Carrier anpassen
FlowStation gleicher Stand	RTP vor <code>media_ready</code> abholen, begrenzter Downlink-Puffer mit <code>dl_jitter_ms</code>	Übernahmekandidat: NetCore überspringt Empfang vor <code>media_ready</code> ; aktuelles Latenzproblem damit noch nicht gemessen
FlowStation gleicher Stand	Statusbefehle quittieren und 30 Sekunden anhand Quelle/Status deduplizieren	Übernahmekandidat: NetCore führt den eigenen Befehlspfad ohne dieses Fenster aus; Ziel-ISSI unterscheidet sich, daher nicht blind kopieren
FlowStation gleicher Stand	INVITE-CANCEL mit passendem Branch	Im entsprechenden NetCore-Pfad vorhanden; vollständige Dialog-Race-Behandlung getrennt prüfen
BlueStation 09d4e0d9, 09.07.2026, v0.5.10	Channel Allocation bei fragmentiertem Downlink in MAC-END verschieben	Konkreter Rückstand: NetCore hat dort weiterhin <code>chan_alloc_element: None</code> und einen TODO; Parser, Encoder und Scheduler zusammen portieren und testen
BlueStation gleicher Stand	Kleine Restkapazität vor Fragmentbeginn abfangen	Entsprechende Header-/Kapazitätsprüfung in NetCore vorhanden; kein Nachweis, dass das gesamte Upstream-Fragmentierungs-Paket integriert ist

Quelle und Stand	Geprüfter Unterschied	Bewertung für NetCore Tetra
Nexus BS ae234dd9, 28.06.2026	Zentrale Dashboard-Auth- Gate-Logik mit Tests für API, WebSocket und Assets	In NetCore nicht in dieser Form vorhanden; an bestehende Maschinen-APIs, Medienexport und Cookie-Anmeldung angepasst prüfen
Nexus BS gleicher Stand	RF-Kalibrierung und gemessene EVM	Weiterer Prüfpunkt; in diesem Lauf keine vollständige Äquivalenzprüfung oder Hardwareabnahme durchgeführt

Priorität haben Circuit-Reopen, RTP-Puffer, MAC-Fragmentierung und die Auth-Grenzen. Jede Übernahme benötigt einen eigenen Code-PR mit Regressionstest; dieser Dokumentationslauf ändert keine Funk- oder Backend-Implementierung.

Vergleichsquellen: FlowStation, BlueStation, Nexus BS. Relevante Quellorte sind `umac/umac_bs.rs`, `umac/subcomp/bs_frag.rs`, `net_asterisk`, `cmce/subentities/sds_bs.rs` und `net_dashboard/server.rs` unter `crates/tetra-entities/src/`.

Anhang C Roadmap und Releases

Stand 27. September 2026 · Grundlage `main` bei `086a81fa8820ef579c475a65a38e3d23644c52f0`.

Diese Seite führt die im Repository dokumentierte bisherige Roadmap, alle 21 veröffentlichten Release-/Archiv-Einträge und die festgehaltenen Ausbauziele zusammen. Historische Planungen werden als solche erhalten. „Im Code vorhanden“ bedeutet keine abgeschlossene Labor- oder On-Air-Abnahme. Es werden keine unbeschlossenen Release-Termine vergeben.

Releaseverlauf

Datumsangaben verwenden Europe/Berlin. Bei Releases ohne Beschreibung wird kein Funktionsumfang hinzuerfunden.

Datum	Release	Dokumentierter Inhalt
26.06.2026	v1.0.0	Hytera-Kompatibilität und Integration des Upstream-Stands 0.3.7.
26.06.2026	v1.0.1	Asterisk/SIP-CANCEL bei Release während des Klingelns korrigiert.
26.06.2026	v1.1.0	Lokales Geräteverzeichnis, lesbare Namen und Karten aktuell registrierter Geräte.
27.06.2026	v1.1.1	Sicherung vor Statusbearbeitung; keine detaillierten Release Notes.
27.06.2026	v1.1.2	Directory-Statuslabels, Home Mode Display PID 220 und begrenztes Replay.
27.06.2026	v1.2.0	Fahrzeug-/Statusgruppen, Live-Directory-Sync und gecachte Gruppenzuordnung.
27.06.2026	v1.2.1	Notruf-Release laut Releasetitel; weitergehender Umfang nicht separat beschrieben.
28.06.2026	v1.3.0	Dual Carrier, konfigurierbare Mittenfrequenzen und Dashboard-Erweiterungen.
28.06.2026	v1.3.1	Kleinere Fehlerkorrekturen laut Release Notes.
30.06.2026	v1.3.2	Fehlerkorrekturen und Neuvergabe von Zeitschlitten.
19.07.2026	v1.4.0	Aufzeichnung von Simplex- und Duplexrufen.
19.07.2026	v1.5.0	Audio-Zentrale, WAV/MP3, Aussendung und NFS-Archiv.
20.07.2026	v1.6.0	Text to Speech für automatisierte Rufe.
21.07.2026	v1.6.1	Kleinere Fehlerkorrekturen.
22.07.2026	v1.7.0	Core-SNDP-Funktionen laut Releasetitel.
30.07.2026	v1.8.0	Release vorhanden; keine detaillierte Beschreibung veröffentlicht.
18.09.2026	v1.9.0-beta.1	MQTT/IoT, zentraler SIP Switch, lokaler Fallback und erweiterte Netzfunktionen.
27.09.2026	v1.9.0	NINA/KATWARN, eigene Warnungen, GPS-SDS, Control-Room-Telemetrie sowie Funk-/SDS-/SIP-Korrekturen.

Branches und erhaltene Historie

Beim Abruf existiert als Remote-Branch nur `main`. Die früheren Branches `mqtt`, `katwarn/nina` und `main_old` sind deshalb keine aktuell zusätzlich prüfbaren Entwicklungsbranches. Archiv-Tags erhalten die alten Stände und ursprünglichen Historien. Der neu angelegte `main` enthält v1.9.0; ein alter lokaler Clone kann

eine abweichende Historie besitzen. Vor einem Update lokale Änderungen sichern und den Release gezielt in einem frischen Clone oder per explizitem Fetch/Detached-Checkout bereitstellen. Kein erzwungener Pull als Standardverfahren.

Archiv	Gesicherter Stand
archive-pre-contributor-reset-2026-09-26	Projektstand vor Neuauflistung; 32af356ec276dd651f9e6be4d05f2baeb5e2d100.
archive-katwarn-nina-2026-09-27	Ursprünglicher Warn-Entwicklungsbranch; 32e0e87d0994107dd007a541df308eae7c691ed2.
archive-main-old-2026-09-27	Alter main bei v1.8.0; f0a4ae39ba8d7d54597732de09bf2025ba45b403.

Ursprüngliche technische Phasen

Die Reihenfolge folgt `system-backend/roadmap.md`; spätere Umsetzungsnotizen konkretisieren ältere Planungstexte.

Phase	Bereich	Umfang	Stand
0	Stabilisierung	Explizite Zustände, Parserrobustheit, Golden Vectors und messbare Fehlerpfade.	Code und Tests vorhanden; kein vollständiger Feldnachweis.
1	TLMC	Configure, Select, Scan, Monitor und Cell Read vollständig durch den Runtime-Pfad führen.	Foundation-Paket C als umgesetzt dokumentiert; Abnahme je primitiver Operation.
2	TLPD	Packet-Primitiven, Downlink, mehrere SNDP-Kontexte sowie Break/Resume.	Foundation-Paket D als umgesetzt dokumentiert; reale Geräteprobe offen.
3	MLE und Zellwechsel	Cell Selection/Reselection, angekündigter und unangekündigter Wechsel, Typ 1/2, Dual Watch.	Teilimplementierungen; Endgeräte-Interoperabilität separat nachweisen.
4	MM und Mobility	Registrierung, Migration und Kontextübergabe zwischen Zellen.	Mobility Core vorhanden; Gesamtpfad nicht als vollständig abgenommen behandeln.
5	CMCE Restore	Lokale/globale Call-IDs, Call Legs, Wiederherstellung und Recovery.	Restore-Pfade vorhanden; laufender Mehrzellenruf bleibt offene Abnahme.
Gate 1	Lokale TBS	Lokaler Betrieb einschließlich Sprache, SDS, Paketdaten und Fehlererholung.	Freigabekriterium; keine automatisch erreichte Vollkonformität.
6	Edge Core Vertrag	Gemeinsame Typen und getrennte Control-, Event- und Media-Verträge.	Vertrag netcore.v1, Gateway und Shared-Komponenten vorhanden.
7	Erste Core Dienste	Node Gateway, Subscriber Core, Group Core und Mobility Core.	Im aktuellen Inventory; frühere Datenbankideen sind keine aktuelle Installationspflicht.
8	Call Control	Netzweite Rufzustände, Floor, Teilnehmer und Call Legs.	Dienst vorhanden; aktuelle Persistenz-/Readiness-Korrektur in v1.9.0.
9	Medien und Recorder	Media Switch, zeitkritische Frames und passive Aufzeichnung.	Dienste vorhanden; Audio in beiden Richtungen und Recovery messen.
10	SDS Router	Zentraler Versand, Store-and-forward, Status und Anwendungsrouten.	Vorhanden; v1.9.0 ergänzt robuste Kurzstatus- und Warn-Idempotenzpfade.
11	Packet Data	SNDP, PDP/NSAPI, IPv4 und IP Gateway.	Dienste vorhanden; Endgeräte-, TUN-, Routing- und Rückwegtest erforderlich.

Phase	Bereich	Umfang	Stand
12	Security und KMF	Policy, Authentisierung, Schlüssel und OTAR.	Open-Lab-/Testprovider; keine belegte produktive TETRA-Authentisierung oder vollständige OTAR-Abnahme.
13	Supplementary Services	Unter anderem DGNA, Prioritäten und ergänzende Dienstmerkmale.	Teilweise vorhanden; jeder Dienst braucht einen eigenen Nachweis.
14	Leitstelle und Anwendungen	Control Room, Observability, Application Gateway und Media Library.	Dienste vorhanden; umfangreicher Arbeitsplatz-Ausbau bleibt geplant.
15	Region und Transit	NetCore-native Regionalvermittlung sowie spätere ETSI-ISI-Anbindung.	Transit vorhanden; standardkonforme ETSI ISI ist ein weiterführendes Ziel.

Umsetzungspakete und späterer Netzausbau

Die Foundation-Pakete A bis E sind in der Roadmap als umgesetzt markiert: A Inventur vom 22.07.2026, B gemeinsame Typen, C TLMC Runtime, D TLPD Runtime, E Tests und Robustheit. Diese Markierungen beschreiben den Entwicklungsstand; sie ersetzen keine Betriebsabnahme. Spätere Pakete erweitern die Core-Dienste um IP Gateway, Security/KMF, Transit, Application Gateway, Media Library, Control Room und Observability. Die aktuelle Inventurliste umfasst 25 deploybare Dienste und eine gemeinsame Bibliothek.

Die MQTT-/IoT-Folgeplanung beginnt mit zentraler Mobility-Wahrheit, Ereignisvertrag, IoT Gateway, Command/Ack-Ledger und Home-Assistent-/Homematic-Anbindung. Paket R beziehungsweise Phase 5 ist in der Roadmap als umgesetzt dokumentiert. Paket S führt Hardware Gateway, RF Monitor und Alarm Workflow in den Phasen 6 bis 8 ein. Paket T beziehungsweise Phase 9 ergänzt WAP-Formulare und strukturierte Aufträge im Task Workflow. Phase 10 ergänzt Asset-, Geräte- und Benutzerverwaltung. Phase 11 bringt den zentralen SIP Switch; im Modus `edge_media` bleiben Medien am lokalen TBS-Asterisk, der zentrale Switch entscheidet über die Serving-TBS und den PBX-Fallback.

v1.9.0 ergänzt die Warnzentrale als 25. Dienst, den GPS-basierten Warnpfad und die zugehörigen Betriebsreparaturen. Weitere Voice-Gateways wie Zello/FRN, aktive LIP-Anforderungen und zusätzliche Karten-/Leitstellenfunktionen sind weiter zu konkretisieren. Vorhandene Konnektoren belegen keine vollständige Ende-zu-Ende-Funktion einer realen Anlage.

Festgehaltene Ausbauziele

Ziel	Geplanter Umfang	Noch erforderliche Entscheidung oder Abnahme
Zentraler Imagebuilder und Pi Imager	Pro TBS erzeugtes, herunterladbares Pi-Image mit passender Konfiguration, VPN/Schlüsseln und Rückweg	Reproduzierbarer Build, sichere Personalisierung, Signatur, Recovery
Wizard Neue TBS	Name, MCC, MNC, ISSI, LA und CC als wenige zentrale Eingaben	Eindeutige IDs, Standortvalidierung und Verhalten ohne Zentrale
Gegenseitige Auto-Discovery	Dienste finden einander im selben VLAN; zusätzlicher Suchknopf pro Dienst	Identitätsprüfung, Konfliktlösung, Ablaufzeit und Suchbereich
Virtuelle Funkgeräte und TBS	Bedienbare PTT-/Display-/Tastenmodelle, GPS und Zellwechsel-Szenarien	Simulatorvertrag und klare Trennung von realen Funkteilnehmern
Laufender Mehrzellenruf	Kontextwechsel, Restore, Medienübergang und SIP-Dialogübernahme	Zwei reale TBS und Funkgeräte; Mid-Call-Handover bisher nicht vollständig belegt

Ziel	Geplanter Umfang	Noch erforderliche Entscheidung oder Abnahme
Control-Room-Arbeitsplatz	Dispatch-Kern, Desktop-Audio, Operator-Identität, Rollen, privilegierte Administration, NFC/RFID und AD	Autoritätsmodell, Rechte, Audio-Latenz; skalierbar von kleinem Touchscreen bis großer Anzeigewand
GPIO und Rack-Platine	Sensoren/Aktoren, I2C-Anzeige, LEDs, Lüfter, Temperatur/Spannung und Watchdog	Pinbelegung, Hardwareprüfung und fertigungstauglicher PCB-Entwurf
HA und Homematic	Quittierte, begrenzte Aktionen aus Funkereignissen	Reale Automationen, Default-Deny und vollständiger SDS-/MQTT-Rückweg
Betriebsreife	Einheitliche Authentisierung, TLS, Rollen und belastbare Abnahme	Kein pauschaler Produktions- oder ETSI-Konformitätsnachweis vorhanden

Nächste sinnvolle Arbeitsschritte

1. Audit an 25 Dienste anpassen und Health-/Metrics-/OpenAPI-Verträge der Warnzentrale bewusst festlegen; Konfigurationspfade und Gateway-Adressen vereinheitlichen.
2. v1.9.0 an einer kontrollierten Anlage abnehmen: Warnung, GPS-Alter, ausgeschlossene Geräte, TBS-Ack, Ablauf und Neustart ohne doppelte Aussendung.
3. SDS-/Call-Control-Readiness messen, kurze Statusmeldungen prüfen und den Sepura-/Frame-18-Vergleich mit identischem Funkaufbau dokumentieren.
4. Die konkreten Upstream-Kandidaten in getrennten Code-PRs mit Regressionstests bearbeiten.
5. Erst danach Mehrzellenbetrieb, Imagebuilder/Discovery und umfangreichen Leitstellen-Arbeitsplatz mit eigenen Abnahmekriterien ausbauen.

Quellen und Pflege

Maßgeblich sind die Repository-Roadmap, die Release-Liste, das Inventory und die konkreten Implementierungen des festgehaltenen Commits. Die Ausbauziele sind Planung, kein zugesagter Releaseumfang. Einträge werden bei neuen Belegen fortgeschrieben, historische Releases bleiben erhalten.